

Table of Contents

Table of Contents	1
はじめに	4
第 I 部 開発編：1つの週末で Google Map 地図ゲームを作る	6
第 1 章 地図ゲームアプリという企画の魅力	6
1-1. なぜ「地図×ゲーム」なのか	6
1-2. 個人開発・単一週末開発に向いている理由	6
1-3. 本書で作るアプリの全体像	7
1-4. 企画を固めるための 3 つの問い	7
第 2 章 「1つの週末」で作ってリリースする実践フロー	8
2-1. なぜ「複数の週末」ではなく「1つの週末」なのか	8
2-2. この週末で「リリース」と呼べる状態の定義	8
2-3. スコープを削る：コア MVP の絞り込み	9
2-4. 全体タイムライン：金曜 21:00～月曜 6:00	9
2-5. タイムボックス法：詰まった時間を切り上げるルール	10
2-6. 技術選定の指針：枯れた技術を選ぶ	11
2-7. 事前準備：金曜の夜を迎える前にやっておくこと	11
第 3 章 開発環境を整える（金曜夜）	13
3-1. 必要なツール	13
3-2. XcodeGen でプロジェクトを構成する理由	13
3-3. Google Maps SDK の導入と API キー取得	13
3-4. 最初の週末のゴール：現在地を表示する	14
第 4 章 古地図・画像オーバーレイの実装（土曜朝）	15
4-1. GMSGroundOverlay の基本	15
4-2. 位置合わせ（ジオリファレンス）の考え方	15
4-3. 古地図カタログの設計	16
4-4. 「統合済みマップ ID」の考え方	17
4-5. 画像サイズとテクスチャアトラスの制約に注意	17
第 5 章 位置情報記録とウォーキングゲーム化（土曜午後）	18
5-1. 徒歩ルートの記録	18
5-2. 軌跡の見た目を整える：Chaikin のコーナーカット法	18
5-3. 過去の軌跡と現在の軌跡を描き分ける	19
5-4. 記録開始時に古地図を自動表示する	19
第 6 章 ゲームフィケーション設計：チェックポイントと御朱印（土曜夜）	21
6-1. 史跡チェックポイントの定義	21
6-2. ゲームフィケーション要素の設計原則	21
第 7 章 AI によるコンテンツ生成（日曜朝）	23
7-1. なぜ AI 生成が地図ゲームと相性が良いのか	23
7-2. API キーの安全管理	23
7-3. プロンプト設計の実例	23
第 8 章 データ永続化とクラウド同期（日曜午前）	24
8-1. Firebase の導入：認証・データベース・ストレージ	24
8-2. 同期のタイミング設計	25
第 9 章 Apple Watch 対応（拡張ステップ：次の週末）	26
9-1. Watch 単体アプリという選択	26
9-2. WatchConnectivity による iPhone との連携	26
9-3. 「連動状態への追いつき」問題	27
9-4. 歩数の正確性を上げる：HealthKit との統合	27

第 10 章 Web 版アプリへの展開（日曜 14:00～18:00）	29
10-1. なぜ Web 版を作るのか	29
10-2. Vite + React + Firebase JS SDK の構成	29
10-3. 未サインインでも見られる「公開ページ」の設計	29
10-4. GitHub Actions による CI/CD の自動化	30
第 11 章 ソーシャル機能とバイラリティの設計	31
11-1. いいね・コメント機能	31
11-2. なぜソーシャル機能が収益化に直結するのか	32
11-3. 共有しやすい導線を作る	32
第 12 章 テスト・デバッグ・リリース準備（日曜夜～月曜朝）	33
12-1. テストすべき優先順位	33
12-2. TestFlight による実地テスト	33
12-3. App Store 申請時の注意点	34
12-4. 月曜早朝：リリース完了の最終チェックリスト	35
第 II 部 収益化・マーケティング編	37
第 13 章 地図ゲームアプリの収益化モデルを選ぶ	37
13-1. 収益化の 4 つの型	37
13-2. おすすめの組み合わせ：フリーミアム＋地域パック課金	37
13-3. StoreKit でのアプリ内課金実装の要点	37
13-4. 「無料枠」の設計は慎重に	38
第 14 章 ASO（App Store 最適化）の基本	38
14-1. アプリ名・サブタイトルの設計	38
14-2. スクリーンショット、動画は「体験」を見せる	38
第 15 章 低予算マーケティング戦略	39
15-1. 個人開発者が使える広報チャンネル	39
15-2. 「Before/After」コンテンツの作り方	39
15-3. コミュニティ・ローカルパートナーシップ	40
15-4. ローンチ時の「初速」を作る施策	40
第 16 章 グロースサイクルと継続的改善	40
16-1. 計測すべき指標	40
16-2. リテンションを上げる小さな改善の積み重ね	41
16-3. コンテンツの継続的な追加	41
16-4. 価格・プラン改定のテスト	41
第 17 章 ケーススタディ：週末開発から一つのアプリができるまで	41
17-1. 最小限のプロトタイプ期	42
17-2. コア体験の拡張期	42
17-3. AI とクラウドによる体験の深化期	42
17-4. エコシステム拡張期	42
17-5. このケーススタディから学べること	42
第 18 章 法務・プライバシー・倫理面の留意点	43
18-1. 位置情報の取り扱い	43
18-2. 著作権・史料の扱い	43
18-3. AI 生成コンテンツの取り扱い	43
18-4. 未成年ユーザーへの配慮	43
第 19 章 よくある質問（FAQ）	44
第 20 章 3 人の起業家哲学から学ぶ収益化・マーケティング戦略	46
20-1. スティーブ・ジョブズ型：「体験の完成度」で無料化を拒否する	46
20-2. ビーター・ティール型：「独占できるニッチ」から始めて拡大する	47
20-3. イーロン・マスク型：「ミッション」と「第一原理」でファンを動員する	49

20-4. 3つの哲学の使い分けと組み合わせ方	50
20-5. Komap 適用チェックリスト	51
20-6. 収益化する方法一覧：3人の哲学から導く具体的なマネタイズ手法	53
第21章 費用構造とマーケティングROIの詳細分析	54
21-1. コスト構造の全体像：固定費 vs 変動費	54
21-2. Google Maps Platform 費用の内訳	55
21-3. Firebase 費用の内訳	56
21-4. AI 生成コストの内訳	58
21-5. ユーザー数別・月額総コストシミュレーション	59
21-6. マーケティング費用対効果（ROI）の分析	59
21-7. マーケティング施策ごとのROIまとめ	61
21-8. Komap 適用：費用・ROIシミュレーションの実践チェックリスト	62
第22章 外部機器連携・写真管理・表示品質の磨き込み	63
22-1. 連携カメラ・連携プリンター：外部のIoT機器と連携する	63
22-2. 投稿写真の管理：フリック・削除・非公開	64
22-3. 地図表示の継続的な磨き込み	65
22-4. Web版をマーケティングの入り口にする	66
22-5. パフォーマンスの継続的な改善——「動いているから十分」で終わらせない	66
22-6. 旅行記機能：既存のAI生成資産を再利用する設計	69
22-7. 本章から学べること	72
付録A 用語集	74
付録B 開発チェックリスト	74
付録C 参考リソース	75

週末だけでできる！Google Map を使った地図 ゲームアプリを作る & 収益化する方法

はじめに

「アプリを作ってみたい。でも本業もあるし、まとまった時間なんて取れない」——多くの個人開発者が最初にぶつかる壁は、技術力ではなく「時間」です。

本書は、この壁を「**たった1つの週末**」——**金曜の夜から月曜の朝まで**という区切られた時間の中で乗り越えるための実践ガイドです。題材として取り上げるのは、Google Map と画像レイヤー（古地図や独自マップ）を組み合わせた「地図ゲームアプリ」。現在地に応じて古い地図や創作の世界地図を重ね合わせ、歩くこと自体をゲームにする——という、位置情報×ゲーミフィケーションの定番かつ拡張性の高いジャンルです。

「週末だけ」という言葉には2つの意味を込めています。1つは「1回の週末（金曜夜～月曜朝）だけで、企画からコア機能の実装、そして実際のリリース（Web 公開ページの本番公開、iOS 版の TestFlight 配布と App Store 審査提出）までを完走する」という短距離走としての意味。もう1つは、そこで生まれたリリース済みのアプリを、その後の週末を使って少しずつ機能拡張していく——という長距離走としての意味です。本書は、この「最初の1週末で世に出し切る」という短距離走の設計を第2章で徹底的に解説し、そのうえで「2つめ以降の週末」で Watch 対応・ソーシャル機能・収益化モデルを積み増していく流れを第3章以降で扱います。

筆者は実際に、Google Maps SDK・古地図オーバーレイ・GPS による徒歩記録・チェックポイント（御朱印）収集・AI による物語生成・Firebase を使ったクラウド同期・Apple Watch 連携・Web 版展開までを、週末単位で1つずつ実装してきました。本書の技術パートは、この実体験にもとづく「実際に動いたコード」と「実際にハマった落とし穴」を土台にしています。

本書は大きく2部構成です。

- **第Ⅰ部（開発編）**：まず「最初の1つの週末（金曜夜～月曜朝）でコア MVP を作り切り、実際にリリースする」ための時間割と実装手順を第2章で詳細に解説したうえで、企画・Google Maps SDK 導入・画像オーバーレイ・位置情報記録・ゲーミフィケーション・AI 連携・データ永続化とクラウド同期・Watch 対応・Web 版展開・テスト・App Store 申請までを、それぞれの技術トピックとして深掘りします。
- **第Ⅱ部（収益化・マーケティング編）**：個人開発の地図ゲームアプリが現実的に収益を得るためのモデル選定、ASO（App Store 最適化）、SNS とコミュニティを使った低予算マーケティング、費用構造とマーケティング ROI の詳細分析、リリース後のグロースサイクル、そして実例に基づくケーススタディを扱います。

地図アプリのイメージ画面



「金曜の夜に手を付け、月曜の朝までには App Store の審査に提出し、Web 版は本番で公開されている」——そんな状態を目指す方に向けて書きました。プログラミング経験はあるが位置情報アプリや Google Maps SDK は初めて、という読者を主な対象としています。企画やマーケティングの章は非エンジニアの方にも読めるように構成しています。

それでは、最初の——そして本書全体の起点となる——1 つの週末から始めましょう。

第 I 部 開発編：1 つの週末で Google Map 地図ゲームを作る

第 1 章 地図ゲームアプリという企画の魅力

1-1. なぜ「地図×ゲーム」なのか

位置情報を使ったアプリは、スマートフォン向けアプリの中でも特に「体験の質」で差別化しやすいジャンルです。理由は 3 つあります。

1. **現実世界がそのままコンテンツになる**：ゲーム内の「マップ」を自分で作り込まなくても、ユーザーが実際に歩く道・訪れる場所がそのままステージになります。開発者が用意するのは「その場所に何を重ねるか」というレイヤーだけで済みます。
2. **外出という行動そのものに価値がある**：健康志向、観光需要、地域活性化といった社会的な文脈と相性が良く、自治体や観光協会との協業、地域メディアでの紹介など、広告費をかけずに露出を得られる経路が多い。
3. **「歩く→何かを集める」というループがゲームとして完成している**：位置情報ゲームの王道パターン(ポケモン GO など) がすでに市場で実証済みであり、ユーザー教育コストが低い。

本書で扱う「古地図オーバーレイ×ウォーキング記録」という切り口は、この中でも特にニッチかつ差別化しやすい領域です。史跡・郷土史・観光といった文脈に接続でき、かつ実装難易度は据え置きのまま「意味のある体験」を作れます。

1-2. 個人開発・単一週末開発に向いている理由

大規模なゲーム開発とは異なり、地図ゲームアプリは次の点で個人開発と相性が良好です。

- **コンテンツ制作がスケーラブル**：3D モデルやアニメーションを大量に作る必要がなく、地図画像・史跡データ・簡単なテキスト（AI で生成可能）でコンテンツを増やせる。
- **サーバーサイドを自前で持たなくてよい**：Firebase（Firestore・Authentication・Storage・Cloud Functions）を使えば、バックエンドエンジニアリングの多くを外部サービスに委譲。
- **プラットフォームの地図基盤が使える**：Google Maps SDK や Apple Maps など、地図描画・現在地取得・ジオコーディングといった重い実装は SDK が担ってくれる。
- **段階的にリリースできる**：「現在地に史跡ピンを立てるだけ」の最小版から始めて、古地図オーバーレイ、ゲーミフィケーション、AI 生成、クラウド同期、Watch 対応、Web 版と、機能を積み増していくロードマップが自然に描ける。
- **1 つの週末（72 時間弱）で「動く・公開されている」状態まで到達できる規模感**：地図表示・オーバーレイ・GPS 記録・チェックポイント・簡易 AI 生成・最小限のクラウド同期・Web 公開ページという核となる機能群は、範囲を 1 エリア・1 古地図に絞り込めば、金曜夜

から月曜朝までの時間内に実装からリリースまで収め切れるボリュームに収まる。

1-3. 本書で作るアプリの全体像

本書を通して組み立てていくアプリは、次のような機能を持ちます（実際に本書の著者が開発したアプリの構成に基づいています）。

- 現在地を Google Map 上に表示し、古地図（またはオリジナルのファンタジーマップ・観光マップなど）をオーバーレイ表示
- スライダーで「現在の地図」⇄「古地図」の透過度を調整
- 地図上のスポットをタップすると、AI がその場所にまつわる物語を生成
- 「スタート」で GPS による徒歩ルートの記録を開始し、「完了」で保存
- 史跡チェックポイントに近づくと「御朱印」（コレクションアイテム）を自動獲得
- 徒歩中に自由なタイミングで写真を投稿できる
- Google サインインでクラウド同期し、Web ブラウザからも記録を閲覧
- Apple Watch 単体でも記録可能
- いいね・コメントなどのソーシャル機能

すべてを最初の週末で作る必要はありません。このうち、現在地表示・古地図オーバーレイ・GPS 記録・チェックポイント（御朱印）・簡易 AI 生成・最小限のクラウド同期・Web 公開ページまでを「最初の 1 つの週末（金曜夜～月曜朝）」で作り切ってリリースする **コア MVP の範囲** とし、Apple Watch 対応・写真投稿・いいね/コメントなどのソーシャル機能は「2 つめ以降の週末」で拡張していく **追加機能の範囲** として、本書は章を並べています。読者は自分のアイデアに合わせて、必要な章だけをつまみ食いしても構いません。

1-4. 企画を固めるための 3 つの問い

開発に入る前に、次の 3 つの問いに答えておくと、後の設計判断がぶれません。

1. **誰が、どんな時に使うのか**（例：休日に散歩する人、旅行先で観光する人、子どもと一緒に地域の歴史を学びたい親）
2. **地図に重ねる「レイヤー」は何か**（古地図、架空の世界地図、スタンプラリー用のイラストマップ、ハザードマップなど）
3. **「集める」対象は何か**（史跡の御朱印、キャラクター、写真、称号、バッジ）

本書のサンプルでは「古地図」「御朱印」「AI が生成する物語」という組み合わせを採用していますが、たとえば「アニメの聖地巡礼マップ」「町内会のスタンプラリー」「防災訓練用のハザードマップワーク」など、同じ技術基盤で全く異なる企画に転用できます。

第2章 「1つの週末」で作ってリリースする実践フロー

本章は本書全体の設計図にあたる最も重要な章です。ここで示すのは「金曜の夜から月曜の朝まで」という1本のタイムラインに、企画・実装・テスト・リリースのすべてを収め切るための具体的な時間割です。第3章以降はこの時間割の各ブロックを技術的に深掘りする構成になっています。

2-1. なぜ「複数の週末」ではなく「1つの週末」なのか

分割された複数の週末（毎週末2〜3時間ずつ、数ヶ月かけて）でアプリを組み立てる進め方には、大きな落とし穴があります。**前回どこまでやったかを思い出すコストが、毎回の作業時間を圧迫する**という問題です。1週間空くと、変数名の付け方、実装途中のTODO、なぜその設計にしたかという判断の理由まで忘れてしまい、再開のたびに「思い出す時間」に30分〜1時間を溶かしてしまいます。数ヶ月に及ぶプロジェクトでは、この「思い出しコスト」の累計が、実際のコーディング時間を上回ることにすらあります。

さらに深刻なのは、**途中で企画そのものへの熱量が冷めてしまう**ことです。作りかけのアプリが手元に残ったまま数ヶ月が過ぎると、当初のワクワクした気持ちを取り戻すのが難しくなり、そのままフェードアウトしてしまう個人開発プロジェクトは非常に多く存在します。

これに対して、**金曜の夜から月曜の朝まで（正味48〜60時間、実作業時間にして20〜30時間程度）を1本のブロックとして使い切る**という進め方には、次のような利点があります。

- 記憶が新鮮なまま最後まで駆け抜けられるため、「思い出しコスト」がほぼゼロになる
- 「月曜の朝までに世に出す」という締め切りが、スコープを絞り込む強制力として働く
- 熱量が高いうちに公開まで到達できるため、モチベーションが切れる前にゴールに到達
- 実際にリリースされた「動いているアプリ」が手元に残るため、達成感が次のアクション（マーケティング、機能拡張）への燃料になる

本書はこの「1つの週末」を、**最初にコア MVP を作り切ってリリースするための短距離走**と定義します。Apple Watch 対応やソーシャル機能など、この週末に収まりきらない要素は、無理に詰め込まず「2つめ以降の週末」に回します（第9章・第11章で扱います）。1つの週末では完成させることよりも、「**世に出ている状態**」を作れることを最優先にしてください。

2-2. この週末で「リリース」と呼べる状態の定義

「リリース」という言葉を曖昧にしたまま走り出すと、ゴールが見えなくなります。本書では、月曜の朝の時点で次の2つが満たされていることを「リリース完了」と定義します。

1. **Web 公開ページが本番環境（独自ドメインまたは Firebase Hosting の既定 URL）で実際に公開されており、誰でもブラウザからアクセスして、コア体験（古地図オーバーレイの見た目、サンプルの記録）を閲覧できる状態になっている**（Web 版は App Store のような審査が

ないため、月曜朝の時点で「一般公開」まで到達可能です)

2. **iOS アプリが TestFlight で動作確認済みであり、App Store Connect 上で審査に提出 (Submit for Review) されている状態になっている** (審査自体には数時間～数日を要するため、月曜朝の時点のゴールは「審査待ち」の状態です。審査通過・一般公開はその後の数日で自然に到達します)

この2段階の定義により、「Web 版はその場で世界に公開される」「iOS 版は月曜朝の時点で“出走準備完了”の状態になっている」という、現実的かつ達成可能なゴールが設定できます。

2-3. スコープを削る：コア MVP の絞り込み

1つの週末に収めるために、最も重要な作業は「機能を減らす」ことです。次のスコープ削減ルールを、金曜の夜、作業を始める前に必ず紙に書き出してください。

- **古地図は1エリア・1枚だけにする** (カタログ機能・複数枚切り替えは次の週末に回す)
- **チェックポイント (史跡) は3～5箇所だけにする** (コンテンツとしての体験は成立するが、大量のデータ入力に時間を溶かさない数)
- **AI が生成する物語は、その場で動的生成せず、あらかじめ3～5パターンだけ用意しておいたテキスト (または API 呼び出し1回で生成し、そのままキャッシュ) で代替してもよい** (本格的な動的生成・プロンプトチューニングは次の週末で磨き込む)
- **写真投稿・いいね・コメント・Apple Watch 対応は今回のスコープに含めない** (コアループ = 「歩く→重なる古地図を見る→チェックポイントで御朱印を獲得する」が体験できれば、この週末のゴールとしては十分)
- **クラウド同期は「保存のみ」に絞る** (複雑な双方向編集・リアルタイムリスナーは含めない。Google サインイン→Firestore への書き込み→Web 側での読み取り表示、という一方向の流れだけを実装する)

このスコープ削減こそが、本書における「垂直スライス」の実践です。ゲーム開発の世界には「垂直スライス (Vertical Slice)」という考え方があります。機能を横断的に少しずつ作るのではなく、1つの体験を最初から最後まで (浅くてもいいので) 通して作ってしまう、という進め方です。この週末で目指す垂直スライスは次のようなものです。

「アプリを起動する → 現在地が地図に表示される → 1枚だけ古地図が重なって見える → 透過度スライダーで見た目が変わる → 歩いて記録を開始する → チェックポイントに近づくと御朱印を獲得する → 記録を保存すると Web でも見られる」

この一本道が動けば、それだけで「リリースできるプロトタイプ」と呼べます。デザインが粗くても、古地図が1枚しかなくても構いません。

2-4. 全体タイムライン：金曜 21:00～月曜 6:00

以下が、本書全体を貫く 1 つの週末のタイムラインです。各ブロックの詳細な実装内容は、対応する第 3～12 章で解説します。

時間帯	ブロック	やること	対応章
金曜 21:00～ 24:00	金曜夜：環境構築	Xcode プロジェクトの雛形作成、XcodeGen 導入、 Google Maps API キー取得、現在地表示まで	第 3 章
土曜 7:00～ 12:00	土曜午前：コア機能①	古地図 1 枚のオーバーレイ表示、透過度スライダー、位置 合わせの微調整	第 4 章
土曜 13:00～ 18:00	土曜午後：コア機能②	GPS による徒歩ルート記録の開始・停止・保存、軌跡の 描画	第 5 章
土曜 19:00～ 23:00	土曜夜：ゲーミフィケーション	チェックポイント 3～5 箇所の定義、接近判定と御朱印の 自動獲得	第 6 章
日曜 7:00～ 9:30	日曜早朝：AI 生成（簡易版）	物語生成の実装、または事前生成した固定テキストの組み 込み	第 7 章
日曜 9:30～ 13:00	日曜午前：永続化 とクラウド同期	SwiftData でのローカル保存、Firebase 導入、Google サ インイン、Firestore への一方向書き込み	第 8 章
日曜 14:00～ 18:00	日曜午後：Web 公開ページ	Vite+React+Firebase JS SDK での最小限の Web 版構 築、Firestore の読み取り表示	第 10 章
日曜 19:00～ 22:00	日曜夜：仕上げと テスト	実機・TestFlight での動作確認、UI 文言・アイコンなどの 最終調整、スクリーンショット撮影	第 12 章
日曜 22:00～ 24:00	日曜深夜：リリー ス準備	Web 版の Firebase Hosting への本番デプロイ、App Store Connect でのメタデータ入力・プライバシー申告	第 12 章
月曜 5:00～ 6:00	月曜早朝：提出	App Store への審査提出（Submit for Review）、Web 公 開ページの最終確認、SNS での公開告知準備	第 12 章

土曜の朝が早すぎる、日曜の深夜作業が厳しい、といった場合は、当然ながら各自の生活リズムに合わせてブロックの時間帯をずらして構いません。重要なのは「ブロックの順番と、各ブロックで完了させる中身」です。総実働時間の目安は 20～28 時間程度（金曜夜 3 時間、土曜 10～12 時間、日曜 11～13 時間、月曜 1 時間）です。

2-5. タイムボックス法：詰まった時間を切り上げるルール

1 つの週末という限られた時間の中では、「1 つの不具合に何時間も溶かしてしまい、後続のブロックが全部後ろ倒しになる」という事態が最大のリスクです。これを防ぐために、次の **45 分ルール** を徹底してください。

1. 1 つの問題（不具合・分からない実装）に取り組む時間を、最大 45 分に区切る

2. 45 分経っても解決しなければ、いったんその実装を諦め、「**見た目だけそれらしく動いているふりをする**」代替実装（モック・ハードコード・仮の固定値）に置き換えて先に進む
3. その問題は付箋やメモアプリに書き出し、「次の週末に直す課題リスト」に回す
4. 週末全体の中で、後半のブロック、リリース準備の時間だけは絶対に削らない。前半のブロックで時間が押した場合は、真っ先に「スコープをさらに削る」ことで帳尻を合わせる

例えば、古地図の位置合わせがどうしても納得のいく精度にならない場合、45 分格闘したら「多少ずれていても良しとする」と割り切って先に進みます。位置合わせの精度向上は、リリース後にいくらかでも改善できる部分です。一方で、「App Store への審査提出」「Web 版の本番デプロイ」という月曜朝のゴールだけは、何を犠牲にしても死守してください。

2-6. 技術選定の指針：枯れた技術を選ぶ

1 つの週末という制約の中では、「学習コストの高い最新技術」よりも「情報量が多く、詰まった時に自力で解決しやすい枯れた技術」を優先すべきです。本書で採用する技術スタックは、いずれも実務で広く使われ、ドキュメントやコミュニティの情報が豊富なものです。

- **iOS UI:** SwiftUI（宣言的 UI で学習コストが低く、変更が素早く反映される）
- **地図:** Google Maps SDK for iOS（Apple 純正の MapKit より古地図オーバーレイやカスタマイズの自由度が高い）
- **位置情報:** CoreLocation（標準 API）
- **ローカル保存:** SwiftData（iOS 17 以降の標準永続化フレームワーク）
- **AI:** OpenAI Chat Completions API（ドキュメントが豊富で、gpt-4o-mini のような低コストモデルも選べる）
- **クラウド:** Firebase（Authentication・Firestore・Storage・Cloud Functions・Hosting）を一式で使い、自前サーバーを持たない
- **Web:** Vite + React + TypeScript + Firebase JS SDK
- **プロジェクト管理:** XcodeGen（.xcodeproj を Git 管理せず、YAML から毎回生成することでコンフリクトを避ける）

これらはいずれも「個人開発者が 1 人で全レイヤーを、初めて触ってもその場で動かせる」ことを重視した選定です。目新しい技術に手を出したくなる気持ちを抑え、この週末だけは「枯れた技術で最短距離を走る」ことに徹してください。

2-7. 事前準備：金曜の夜を迎える前にやっておくこと

1 つの週末を最大限に活かすために、金曜の夜が来る前（平日の隙間時間）に済ませておくべき準備が 3 つあります。

1. **Google Cloud Platform アカウントの作成、Google Maps API キーの発行**（審査や承認待ちが発生することは基本的にありませんが、初回のアカウント設定に手間取ることがあるため、平日のうちに済ませておく）
2. **Apple Developer Program への登録**（年会費の支払い、本人確認に数日かかることがあるため、この週末に iOS アプリをリリースする予定であれば、遅くとも前の週のうちに登録を済ませておく）
3. **Firestore プロジェクトの作成、OpenAI（または利用する AI API）アカウントの作成と API キー発行**

これらのアカウント発行・登録作業は、待ち時間が発生する可能性がある「非同期の作業」です。週末の実装時間を、こうした待ち時間で無駄にしないよう、必ず事前に完了させておいてください。

地図アプリのイメージ画面



第3章 開発環境を整える（金曜夜）

3-1. 必要なツール

- Xcode（最新の安定版。iOS 17 以降のシミュレータ/実機が必要）
- Homebrew（Mac のパッケージマネージャ）
- XcodeGen（brew install xcodegen）
- Google Cloud Platform アカウント（Google Maps API キー発行用）
- （後の週末で使用）Node.js、Firebase CLI、OpenAI アカウント

3-2. XcodeGen でプロジェクトを構成する理由

.xcodproj ファイルは XML ベースのバイナリに近い構造を持ち、複数人（あるいは複数の作業セッション）で編集すると Git の差分が読みにくく、コンフリクトも起きやすいという弱点があります。XcodeGen を使うと、プロジェクト構成をシンプルな YAML ファイル（project.yml）で宣言し、コマンド一発で.xcodproj を生成できます。

```
name: Komap
options:
  bundleIdPrefix: com.example
targets:
  Komap:
    type: application
    platform: iOS
    deploymentTarget: "17.0"
    sources: [Komap]
    settings:
      base:
        PRODUCT_BUNDLE_IDENTIFIER: com.example.Komap
    dependencies:
      - package: GoogleMaps
```

.xcodproj は.gitignore に加え、project.yml だけを Git 管理する運用にします。これにより、複数のブランチで作業してもコンフリクトは YAML レベルの軽微なものにとどまります。このコマンドを覚えておくだけで、プロジェクト構成を変更した後も常に最新の.xcodproj を再生成できます。

3-3. Google Maps SDK の導入と API キー取得

1. [Google Cloud Console](#) で新規プロジェクトを作成
2. 「Maps SDK for iOS」を有効化
3. 認証情報から API キーを発行

4. 本番運用時は API キーに「iOS アプリの制限（Bundle ID 制限）」をかけて悪用を防ぐ

API キーはソースコードに直接埋め込まず、.xcconfig ファイルなどビルド設定側で管理し、.gitignore に追加してリポジトリにコミットしないようにします。これはセキュリティ上非常に重要な習慣です。

```
GOOGLE_MAPS_API_KEY = AlzaSyxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Info.plist 側では、この値を\$(GOOGLE_MAPS_API_KEY)のようにビルド変数として参照させ、コード上では Bundle.main.object(forKey:)などを通じて読み込みます。XcodeGen を使う場合、Swift Package Manager 経由で Google Maps SDK を依存関係に追加できます。

```
packages:
  GoogleMaps:
    url: https://github.com/googlemaps/ios-maps-sdk
    exactVersion: 9.0.0
```

3-4. 最初の週末のゴール：現在地を表示する

GMSMapView を SwiftUI から使うには、UIViewRepresentable でラップします。

```
import GoogleMaps
import SwiftUI
struct GoogleMapRepresentable: UIViewRepresentable {
    @Binding var cameraPosition: CLLocationCoordinate2D

    func makeUIView(context: Context) -> GMSMapView {
        let camera = GMSCameraPosition.camera(
            withLatitude: cameraPosition.latitude,
            longitude: cameraPosition.longitude,
            zoom: 16
        )
        let mapView = GMSMapView(frame: .zero, camera: camera)
        mapView.isMyLocationEnabled = true
        mapView.settings.myLocationButton = true
        return mapView
    }
    func updateUIView(_ uiView: GMSMapView, context: Context) {
        // 現在地の更新に応じてカメラを動かす処理をここに書く
    }
}
```

CoreLocation で現在地を取得する LocationManager クラスを作り、CLLocationManagerDelegate 経由で位置情報の更新を受け取ります。プライバシー説明文

(NSLocationWhenInUseUsageDescription) を Info.plist（または XcodeGen の project.yml の info 設定）に忘れず追加してください。これを忘れると、審査どころかシミュレータでも現在地取得が失敗します。

この時点で「アプリを起動すると現在地が地図上に青い点で表示される」状態になっていれば、第 1 週末のゴール達成です。

第 4 章 古地図・画像オーバーレイの実装（土曜朝）

4-1. GMSGroundOverlay の基本

Google Maps SDK には、指定した緯度経度の矩形範囲に画像を貼り付ける GMSGroundOverlay というクラスがあります。これが「古地図を現在の地図に重ねる」機能の核になります。

```
let southWest = CLLocationCoordinate2D(latitude: 35.685, longitude: 139.750)
let northEast = CLLocationCoordinate2D(latitude: 35.695, longitude: 139.765)
let bounds = GMSCoordinateBounds(coordinate: southWest, coordinate: northEast)

let overlay = GMSGroundOverlay(bounds: bounds, icon: UIImage(named: "old_map_edo"))
overlay.bearing = 0 // 画像が北を向いていない場合はここで回転角を指定
overlay.opacity = 0.7 // 透過度 (0~1)
overlay.map = mapView
```

ポイントは次の 3 つです。

- **bounds** : 画像の南西端・北東端の緯度経度。ここが古地図の「位置合わせ」の核心部分です。
- **bearing** : 画像が真北を向いていない場合の回転角。古地図のように傾いた向きで描かれている画像では必須の設定で、これを忘れると「絵柄と実際の地形がずれる」不具合の原因になります。
- **opacity** : 透過度。スライダーの UI とバインドすることで「現在の地図⇄古地図」を自由に行き来できる体験を作ります。

4-2. 位置合わせ（ジオリファレンス）の考え方

古地図の位置合わせは、本格的にやるなら GIS 的な「ジオリファレンス」（地図上の複数の既知地点を基準に、画像の歪みを補正しながら座標を割り当てる作業）が必要ですが、個人開発の週末プロジェクトではそこまで厳密にやる必要はありません。

実務的な近似法は次の通りです。

1. 古地図の画像の中で、現代でも判別できるランドマーク（大きな交差点、寺社、川、皇居のお堀など）を 2~3 点選ぶ
2. Google Map で同じランドマークの緯度経度を調べる
3. 画像の四隅（あるいは南西端・北東端）が、その基準点から見てどの位置に来るかを目分量で算出する
4. 実機・シミュレータで表示して、ずれを見ながら微調整する

この仮の値で始めて、後から精度を上げていく進め方で十分です。実際、本書のベースにしたアプリ

でも「現在の地理に大まかに合わせた仮の座標」であることを明記した上でリリースしています。ユーザー体験としては、多少のズレよりも古地図が見えるという体験の価値の方が大きいからです。

4-3. 古地図カタログの設計

古地図を 1 枚だけでなく複数扱うために、カタログ構造を用意します。

```
struct OldMapEntry: Identifiable {
    let id: String
    let title: String
    let era: String      // 例: "江戸時代 (安政期)"
    let imageName: String
    let southWest: CLLocationCoordinate2D
    let northEast: CLLocationCoordinate2D
    let bearing: Double
}

enum OldMapCatalog {
    static let all: [OldMapEntry] = [
        OldMapEntry(
            id: "edo-castle",
            title: "江戸城周辺",
            era: "安政期",
            imageName: "old_map_edo",
            southWest: .init(latitude: 35.678, longitude: 139.745),
            northEast: .init(latitude: 35.696, longitude: 139.762),
            bearing: 0
        ),
        // ここに古地図を追加していく
    ]
}
```

古地図が 10 枚、20 枚と増えてくると、SwiftUI の Menu のような単純なドロップダウンでは選択肢が画面に収まりきらず、スクロールもできないまま一部の選択肢が表示されなくなる、という問題が起こり得ます（実際に著者もこの問題に遭遇しました）。対策として、.sheet + List によるピッカー画面を用意すると、件数が増えても確実にスクロールで選択できます。

```
struct OldMapPickerSheet: View {
    let entries: [OldMapEntry]
    let onSelect: (OldMapEntry) -> Void
    var body: some View {
        List(entries) { entry in
            Button {
                onSelect(entry)
            } label: {
                VStack(alignment: .leading) {
                    Text(entry.title).font(.headline)
                    Text(entry.era).font(.caption).foregroundColor(.secondary)
                }
            }
        }
    }
}
```

4-4. 「統合済みマップ ID」の考え方

コンテンツを育てていくと、「最初は別々に用意していた 2 つの古地図を、範囲を広げて 1 枚に統合したい」というリファクタリングが発生します。このとき、すでにユーザーの記録に古い ID（廃止したマップ ID）が保存されている場合、その ID を引き続き正しく解決できる仕組みが必要です。

```
extension OldMapCatalog {  
    private static let mergedIdAliases: [String: String] = [  
        "kudanshita-chidorigafuchi": "edo-castle",  
        "kasumigaseki-toranomon": "edo-castle",  
        "ueno-kaneiji": "hongo-yanaka-ueno",  
    ]  
    static func resolve(id: String) -> OldMapEntry? {  
        let resolvedId = mergedIdAliases[id] ?? id  
        return all.first { $0.id == resolvedId }  
    }  
}
```

こうしておけば、過去に保存された時空旅の記録・チェックポイントのデータをマイグレーションすることなく、表示ロジック側で「統合先」を常に正しく解決できます。データベースのマイグレーションは個人開発において地味に時間を奪う作業なので、可能な限り「表示側で吸収する」設計を検討する価値があります。

4-5. 画像サイズとテクスチャアトラスの制約に注意

Google Maps SDK の内部実装には、オーバーレイやマーカーのために使う「テクスチャアトラス」の容量上限があります。フル解像度（3000px 超）の大きな画像を何枚も同時にロードして切り替えていくと、ある時点から古地図が真っ白になったり、一部しか描画されなくなったりする不具合が起きます。対策は次の 2 点です。

1. **単体表示時は画像を縮小してから使う**：長辺 1600px 程度にダウンサンプリングしたキャッシュを作り、それをオーバーレイに使う。
2. **同時に読み込む実画像オーバーレイの枚数を制限する**：「全ての古地図を表示」のような一覧表示機能を作る場合、実画像を使うオーバーレイは同時に 4 枚程度までに制限し、それ以外は簡易表示（枠線や低解像度サムネイル）にとどめる。

```
func downsampledImage(named: String, maxDimension: CGFloat = 1600) -> UIImage? {  
    guard let source = UIImage(named: named) else { return nil }  
    let scale = min(1, maxDimension / max(source.size.width, source.size.height))  
    guard scale < 1 else { return source }  
    let newSize = CGSize(width: source.size.width * scale, height: source.size.height * scale)  
    let renderer = UIGraphicsImageRenderer(size: newSize)  
    return renderer.image { _ in source.draw(in: CGRect(origin: .zero, size: newSize)) }  
}
```

このような「SDK の見えない制約」は公式ドキュメントに明記されていないことが多く、実際に手を動かして初めて発覚します。週末開発では、こうした落とし穴に遭遇したら、まず「同時に扱う量を減らす」という単純な対策から試すのが定石です。

第 5 章 位置情報記録とウォーキングゲーム化（土曜午後）

5-1. 徒歩ルートへの記録

CoreLocation の CLLocationManager を使い、「スタート」ボタンが押されている間、位置情報の更新を蓄積していきます。

```
final class LocationManager: NSObject, ObservableObject, CLLocationManagerDelegate {
    private let manager = CLLocationManager()
    @Published var recordedPath: [CLLocationCoordinate2D] = []
    @Published var isRecording = false

    override init() {
        super.init()
        manager.delegate = self
        manager.desiredAccuracy = kCLLocationAccuracyBest
        manager.allowsBackgroundLocationUpdates = true
        manager.pausesLocationUpdatesAutomatically = false
    }

    func startRecording() {
        recordedPath.removeAll()
        isRecording = true
        manager.startUpdatingLocation()
    }

    func stopRecording() {
        isRecording = false
        manager.stopUpdatingLocation()
    }

    func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [CLLocation]) {
        guard isRecording, let location = locations.last else { return }
        recordedPath.append(location.coordinate)
    }
}
```

バックグラウンドでの位置情報取得を行う場合は、Info.plist に NSLocationAlwaysAndWhenInUseUsageDescription を追加し、Capabilities で「Background Modes」の「Location updates」を有効化する必要があります。バッテリー消費との兼ね合いもあるため、「アプリがアクティブな間だけ記録する」という制約でスタートし、後からニーズに応じて拡張するのが週末開発では現実的です。

5-2. 軌跡の見た目を整える：Chaikin のコーナーカット法

GPS の生データをそのまま線で結ぶと、信号のノイズでジグザグした見た目になります。これを滑らかにする簡易的な方法として、Chaikin のコーナーカット法（角を切り落として丸めるアルゴリズム）が使えます。

```
func chaikinSmooth(_ points: [CLLocationCoordinate2D], iterations: Int = 2) -> [CLLocationCoordinate2D] {
    var result = points
```

```

for _ in 0..

```

このような「見た目の質」に効く小さな改善は、ユーザーが実際にスクリーンショットを SNS でシェアするかどうかに直結します。マーケティング効果を考えると、開発の早い段階で軌跡の見た目にひと手間かけておく価値は十分にあります。

5-3. 過去の軌跡と現在の軌跡を描き分ける

記録中の画面で「保存済みの過去の軌跡」を薄い色で背景に表示し、「今歩いている軌跡」を目立つ色で表示すると、ユーザーは自分の探索の進み具合を直感的に把握できます。ゲームでいう「マップの踏破率」の可視化に相当します。

```

let pastPathColor = UIColor.systemBlue.withAlphaComponent(0.25)
let currentPathColor = UIColor.systemBlue

let pastPolyline = GMSPolyline(path: pastPathGooglePath)
pastPolyline.strokeColor = pastPathColor
pastPolyline.strokeWidth = 3
pastPolyline.map = mapView

let currentPolyline = GMSPolyline(path: currentPathGooglePath)
currentPolyline.strokeColor = currentPathColor
currentPolyline.strokeWidth = 5
currentPolyline.map = mapView

```

5-4. 記録開始時に古地図を自動表示する

ユーザーが「スタート」を押した瞬間に古地図が未選択であれば自動的に表示し、透過度も一定以上に引き上げる、という細かい配慮も体験の質を大きく左右します。これは「わざわざ古地図を選ぶ」という手間を 1 ステップ減らすだけで、ユーザーが本来体験したい「古地図の上を歩いている感

覚」により早くたどり着けるようにする施策です。

```
func startWalking() {  
    if selectedOldMap == nil {  
        selectedOldMap = OldMapCatalog.all.first  
        overlayOpacity = max(overlayOpacity, 0.6)  
    }  
    locationManager.startRecording()  
}
```

こうした「あと一步の親切設計」は、レビューの星評価やリテンション（継続利用率）に直結するため、収益化パートでも改めて触れます。

第6章 ゲームフィケーション設計：チェックポイントと御朱印（土曜夜）

6-1. 史跡チェックポイントの定義

古地図ごとに、実際の緯度経度を持った「チェックポイント」を定義します。

```
struct HistoricSite: Identifiable {
  let id: String
  let mapId: String    // 対応する古地図の ID
  let name: String
  let coordinate: CLLocationCoordinate2D
  let storyPrompt: String // AI に物語を生成させる際のヒント
}
```

ユーザーがチェックポイントに一定距離（例：30 メートル）以内に近づいたら、自動的に「御朱印」を獲得したことにします。

```
func checkProximity(currentLocation: CLLocationCoordinate2D, sites: [HistoricSite], radius: CLLocationDistance = 30) -> HistoricSite? {
  let current = CLLocation(latitude: currentLocation.latitude, longitude: currentLocation.longitude)
  for site in sites {
    let target = CLLocation(latitude: site.coordinate.latitude, longitude: site.coordinate.longitude)
    if current.distance(from: target) <= radius {
      return site
    }
  }
  return nil
}
```

この判定を位置情報の更新のたびに走らせ、まだ獲得していないチェックポイントであれば、獲得演出（トースト通知、効果音、バイブレーション）を出すとゲームらしい達成感が生まれます。

6-2. ゲームフィケーション要素の設計原則

心理学の「オペラント条件付け」や「可変報酬」の考え方をシンプルに応用すると、地図ゲームには次のような報酬設計が有効です。

- **即時報酬**：チェックポイントに近づいた瞬間に得られる御朱印（探索の直接的な報酬）
- **蓄積報酬**：御朱印のコレクション数、歩いた距離の累計、訪れた古地図の枚数
- **創造的報酬**：AI が生成する、その場所固有の物語（毎回異なる体験になり、飽きにくい）
- **社会的報酬**：いいね・コメントなど、他者からの反応（本書後半で詳述）

これら4種類の報酬をバランス良く配置することで、「1回だけ試して終わり」ではなく「継続して使う」アプリになります。特に個人開発の場合、蓄積報酬（コレクション要素）は実装コストが低い

割にリテンション効果が高いため、優先度を上げることをお勧めします。

史跡以外の場所でも、ユーザーが自由なタイミングで写真を投稿できるようにすると、「決められたルート」以外の自由な探索行動も肯定できます。

```
struct WalkPhotoPost: Identifiable {  
    let id: UUID  
    let coordinate: CLLocationCoordinate2D  
    let imageData: Data  
    let capturedAt: Date  
    let isCheckpointPhoto: Bool // 史跡での御朱印用か、自由投稿か  
}
```

PhotosPicker (PhotosUI) を使う場合、Menu の直下に PhotosPicker を直接置くと開かないことがある、という SwiftUI 側の既知の制約があります。回避策として、Button + .photosPicker(isPresented:) モディファイアの組み合わせに変更すると安定して動作します。

```
Button("ライブラリから選ぶ") {  
    isPhotoPickerPresented = true  
}  
.photosPicker(isPresented: $isPhotoPickerPresented, selection: $selectedItem)
```

このような「SwiftUI のフレームワーク特有の罫」は検索してもすぐには見つからないことが多いため、実際に手を動かして初めて得られる知見です。週末開発では、こうした詰まりに時間を溶かさないう、動かない場合はコンポーネントの組み合わせを変える転換を早めに試すことが重要です。

第7章 AIによるコンテンツ生成（日曜朝）

7-1. なぜ AI 生成が地図ゲームと相性が良いのか

古地図や史跡の数だけ、手作業で解説文を用意するのは個人開発者にとって現実的ではありません。OpenAI の Chat Completions API のような LLM（大規模言語モデル）を使えば、「その場所の緯度経度・地名・時代」といった情報をプロンプトに渡すだけで、その場に応じた物語やエピソードを動的に生成できます。

7-2. API キーの安全な管理

API キーをアプリのバイナリに埋め込むと、リバースエンジニアリングで抜き取られるリスクがあります。個人開発の初期段階では次のいずれかの方針を取ります。

- **ユーザー自身に API キーを入力してもらい、端末の Keychain に保存する**（開発初期・検証段階向け。コストをアプリ開発者が負担しない）
- **Cloud Functions 等のサーバーサイド経由で API を呼び出し、キーはサーバー側だけに置く**（本格運用向け。ユーザー体験は良いが、開発者が API 利用料を負担する）

```
import Security
enum KeychainStore {
    static func save(key: String, value: String) {
        let data = Data(value.utf8)
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecValueData as String: data,
        ]
        SecItemDelete(query as CFDictionary)
        SecItemAdd(query as CFDictionary, nil)
    }
}
```

7-3. プロンプト設計の実例

```
func buildPrompt(for site: HistoricSite, era: String) -> String {
    """
    あなたは日本の郷土史に詳しい語り部です。
    以下の場所について、旅人が思わず読み込んでしまうような
    短い物語（200 字程度）を生成してください。
    場所: ¥(site.name)
    時代背景: ¥(era)
    ヒント: ¥(site.storyPrompt)
    文体は柔らかく、断定しすぎない表現（～と伝えられています、など）を使い、
    史実と創作の境界を意識してください。
    """
}
```

```
}

```

「史実と創作の境界を意識する」という一文を入れておくのは、コンテンツの信頼性に関わる重要なポイントです。AI 生成コンテンツは説得力のある文章を作れる反面、事実と異なる内容を断定的に書いてしまうことがあります。アプリ内で「AI が生成した創作を含みます」という注意書きを添えることも、ユーザーへの誠実さとして推奨します。

レイテンシ管理は以下の通りです。

- 低コストなモデル（例：gpt-4o-mini のような軽量モデル）を選び、1 回あたりのトークン数を抑える
- 生成結果をローカル（SwiftData）にキャッシュし、同じ場所・同じユーザーに対して毎回課金しない
- 生成中はローディングインジケータを表示し、体感速度を担保する

週末開発では「完璧な文章生成」よりも「体験として破綻しない範囲でコストを抑える」ことを優先しましょう。

第 8 章 データ永続化とクラウド同期（日曜午前）

8-1. Firebase の導入：認証・データベース・ストレージ

SwiftData（iOS 17 以降の標準永続化フレームワーク）を使うと、Core Data より少ないコードでモデルを永続化できます。

```
import SwiftData
@Model
final class WalkRoute {
    var id: UUID
    var startedAt: Date
    var mapId: String
    var coordinates: [CLLocationCoordinate2D]
    var stepCount: Int
    var notes: String?
    init(id: UUID = UUID(), startedAt: Date, mapId: String, coordinates: [CLLocationCoordinate2D], stepCount: Int) {
        self.id = id
        self.startedAt = startedAt
        self.mapId = mapId
        self.coordinates = coordinates
        self.stepCount = stepCount
    }
}
```

まずクラウド同期なしで「ローカルだけで完結するアプリ」として動く状態を作ることをお勧めします。クラウド連携は後から追加できますが、ローカルで完結する体験の設計を最初にしっかり固めておくと、後の Firebase 連携作業がスムーズになります。

クラウド同期には、個人開発者にとって導入コストの低い Firebase が定番です。

- **Firebase Authentication** : Google サインインなど、OAuth ベースの認証をほぼコード不要
- **Cloud Firestore** : NoSQL 型のリアルタイムデータベース。ユーザーごとのデータを `users/{uid}/walkRoutes` のような階層構造で管理
- **Firebase Storage** : 写真や画像本体の保存に使用
- **Cloud Functions** : TestFlight 招待メールの自動送信などサーバーサイド処理に利用
- **Firebase Hosting** : Web 版アプリの公開先

個人情報や位置情報を扱うアプリでは、セキュリティルールの設計が特に重要です。基本方針は「本人のデータは本人だけが読み書きでき、公開データ（シェアされた記録）は誰でも閲覧できるが書き込みは本人のみ」という形です。

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /sharedTrips/{tripId} {
      allow read: if true;
      allow write: if request.auth != null && request.auth.uid == resource.data.ownerId;
    }
    match /likes/{likeId} {
      allow read: if true;
      allow write: if request.auth != null;
    }
    match /comments/{commentId} {
      allow read: if true;
      allow create: if request.auth != null;
    }
  }
}
```

セキュリティルールは「デフォルトで拒否、必要な範囲だけ許可する」という発想で書くことが鉄則です。個人開発だからといって手を抜くと、後から意図しないデータ漏洩や改ざんのリスクを抱えることになります。

8-2. 同期のタイミング設計

すべての操作を毎回リアルタイムでクラウドに送ると、通信量・API 呼び出し回数（Firestore は書き込み回数に応じて課金される）が増え、無料枠を超えるコストにつながります。実務的には次のような設計が有効です。

- ローカルの保存操作が完了した後、非同期でバックグラウンドアップロードする
- 徒歩ルートの記録中は逐次アップロードせず保存ボタンが押された時にまとめて送信
- 画像はアップロード前にリサイズ・圧縮してから Firebase Storage に送る

第9章 Apple Watch 対応（拡張ステップ：次の週末）

本章の位置づけ：Apple Watch 対応は、第2章で定義した「1つの週末」のコア MVP には含めていません。Watch 単体アプリの実装・WatchConnectivity による同期・復帰時の整合性対応は、それ自体で1回分の週末に匹敵するボリュームがあるためです。最初のリリースをまず月曜朝に終わらせ、ユーザーの反応を見てから、2つめの週末でこの章に取り組むことを推奨します。

9-1. Watch 単体アプリという選択

Apple Watch 向けの実装には、「iPhone アプリのコンパニオン（付属）として動く Watch アプリ」と「Watch 単体でも起動・記録できるアプリ」の2種類があります。ウォーキングゲームでは、iPhone をポケットにしまったまま、Watch だけで記録を開始・終了できる体験の価値が高いため、単体アプリとしての実装をお勧めします。HKWorkoutSession を使うことで、Watch 単体でも GPS による位置情報取得とワークアウト記録（Apple Health への「ウォーキング」ワークアウトとしての保存）が可能になります。

```
import HealthKit
import CoreLocation
final class WatchWorkoutLocationTracker: NSObject, HKWorkoutSessionDelegate {
    private let healthStore = HKHealthStore()
    private var session: HKWorkoutSession?

    func startWorkout() {
        let config = HKWorkoutConfiguration()
        config.activityType = .walking
        config.locationType = .outdoor

        session = try? HKWorkoutSession(healthStore: healthStore, configuration: config)
        session?.delegate = self
        session?.startActivity(with: Date())
    }
}
```

9-2. WatchConnectivity による iPhone との連携

Watch で記録した軌跡・獲得した御朱印・投稿写真を、iPhone 側にもリアルタイムで反映させるには、WatchConnectivity フレームワークの WCSession を使います。

```
import WatchConnectivity
final class WatchConnectivityManager: NSObject, WCSessionDelegate {
    static let shared = WatchConnectivityManager()
    private let session = WCSession.default
    func activate() {
        guard WCSession.isSupported() else { return }
        session.delegate = self
        session.activate()
    }
}
```

```

func sendSnapshot(_ payload: [String: Any]) {
    session.sendMessage(payload, replyHandler: nil, errorHandler: nil)
}
}

```

9-3. 「連動状態への追いつき」問題

Watch 単体で記録を開始した後に iPhone 側アプリを起動した場合、sendMessage によるリアルタイム通知だけに頼っていると、次の更新が届くまで iPhone 側は「連動中」と認識できません。この問題への対策として、WCSession が有効化されたタイミングで、保存済みの applicationContext（直近の状態のスナップショット）を読み直す実装を入れておくと、アプリを開いた直後から正しい連動状態を表示できます。

```

func session(_ session: WCSession, activationDidCompleteWith activationState: WCSessionActivationState, error: Error?) {
    if let context = session.receivedApplicationContext as? [String: Any] {
        applyContext(context)
    }
}

```

さらに、Watch 側は記録終了時に applicationContext を空にしておくことで、「終了済みの記録がまだ続いているように見える」誤表示も防げます。また、古いスナップショット（例：5 分以上前に更新されたもの）は無視するようにガードすることで、通信の遅延によって「記録中」という誤った状態が残り続ける不具合も回避できます。このように、Watch 連携は「ハッピーパスだけでなく、アプリの再起動やバックグラウンド遷移といった中断シナリオ」を丁寧に潰し込む作業が実装コストの大半を占めます。週末開発では、まず単純な連携（片方向の通知）を動かし、その後の週末で「復帰時の整合性」を 1 つずつ潰していくアプローチが現実的です。

9-4. 歩数の正確性を上げる：HealthKit との統合

CMPedometer（Core Motion）だけで歩数を取る場合、iPhone のセンサーのみに依存するため、Apple Watch を装着している場合の歩数と乖離することがあります。HealthKit の歩数データ（Watch・iPhone 双方のセンサー値が統合されたもの）を読み取り専用で利用すると、より正確な歩数を記録に反映できます。

```

import HealthKit
final class HealthKitStepReader {
    private let healthStore = HKHealthStore()

    func requestAuthorization(completion: @escaping (Bool) -> Void) {
        guard let stepType = HKObjectType.quantityType(forIdentifier: .stepCount) else { return }
        healthStore.requestAuthorization(toShare: [], read: [stepType]) { success, _ in
            completion(success)
        }
    }

    func fetchStepCount(from start: Date, to end: Date, completion: @escaping (Int?) -> Void) {
        guard let stepType = HKObjectType.quantityType(forIdentifier: .stepCount) else { return }
        let predicate = HKQuery.predicateForSamples(withStart: start, end: end)
        let query = HKStatisticsQuery(quantityType: stepType, quantitySamplePredicate: predicate, options: .cumulativeSum) { _,
result, _ in

```

```
        let steps = result?.sumQuantity()?.doubleValue(for: .count())
        completion(steps.map { Int($0) })
    }
    healthStore.execute(query)
}
}
```

HealthKit を使う場合、NSHealthShareUsageDescription（読み取りの説明文）に加え、実際には書き込みを行わない場合でも NSHealthUpdateUsageDescription を Info.plist に追加しておく必要があります。これを怠ると、App Store Connect へのアップロード自体がエラー90683 のようなメッセージで拒否されることがあります。HealthKit のエンティトルメントや API をリンクした時点で Apple の静的検証が両方の説明文を要求するためです。審査前に必ずチェックしておきましょう。

第 10 章 Web 版アプリへの展開（日曜午後）

本章の位置づけ：第 2 章のタイムラインでは、Web 版は「最小限の公開ページ」としてコア MVP に含めています。Firestore の読み取り表示・公開ページの本番デプロイまでをこのブロックのゴールとし、いいね・コメント等のソーシャル機能（第 11 章）は 2 つめ以降の週末に回してください。

10-1. なぜ Web 版を作るのか

iOS アプリだけで完結させず、同じ Firebase プロジェクトを参照する Web アプリを用意すると、次のメリットがあります。

- ユーザーが自分の記録を PC の大画面で見返せる
- 検索エンジンや SNS のリンクから、アプリをインストールしていない人にも「公開ページ」として体験の一部を見せられる（マーケティング上、非常に強力）
- App Store の審査を経ずに素早く機能追加・修正ができる

10-2. Vite + React + Firebase JS SDK の構成

```
npm create vite@latest web -- --template react-ts
cd web
npm install firebase

// web/src/lib/firebase.ts
import { initializeApp } from "firebase/app";
import { getAuth, GoogleAuthProvider } from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
};

export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const googleProvider = new GoogleAuthProvider();
export const db = getFirestore(app);
```

iOS アプリと Web アプリが同じ Firebase プロジェクトの同じ UID（Google サインインで一意になる）を使うことで、「iOS で記録した時空旅を、Web で即座に閲覧できる」という体験がシームレスに実現します。

10-3. 未サインインでも見られる「公開ページ」の設計

マーケティングの観点で特に重要なのが、サインインしていない訪問者でも一部のコンテンツ（例えば、ユーザーが「公開」設定にした時空旅の記録）を閲覧できるページです。

// 公開設定の sharedTrips だけを取得し、サインインなしで表示する

```
const publicTripsQuery = query(
  collection(db, "sharedTrips"),
  where("isPublic", "=", true),
  orderBy("createdAt", "desc"),
  limit(20)
);
```

さらに、「現在このページを見ている人数」をリアルタイムに表示するような小さな仕掛けを入れると、訪問者に「動いているサービスである」という信頼感を与られます。Firestore の onSnapshot（リアルタイムリスナー）と、一定間隔でのハートビート（自分の存在を presence コレクションに書き込み続ける）を組み合わせれば実装できます。

*// 5 秒ごとに自分の presence を更新し、30 秒以上更新のない presence は
// Cloud Functions の定期実行（もしくはクライアント側のフィルタ）で除外する*

```
setInterval(() => {
  setDoc(doc(db, "presence", sessionId), { lastSeen: serverTimestamp() });
}, 5000);
```

このような「今まさに他の誰かが見ている・使っている」という演出は、後述するマーケティング戦略における「社会的証明（Social Proof）」の一種であり、実装コストの割に訪問者の滞在時間・信頼感に寄与します。

10-4. GitHub Actions による CI/CD の自動化

Web 版の変更を毎回手動デプロイするのは週末開発では手間がかかりすぎます。GitHub Actions を使い、main ブランチへの push をトリガーに Firebase Hosting へ自動デプロイする仕組みを最初期に作っておくと、後の開発速度が大きく変わります。

.github/workflows/deploy-web.yml

```
name: Deploy Web
on:
  push:
    branches: [main]
    paths:
      - "web/**"
      - "firebase.json"
      - "firebase/**"
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
```

```
- run: npm ci
  working-directory: web
- run: npm run build
  working-directory: web
- uses: FirebaseExtended/action-hosting-deploy@v0
  with:
    repoToken: ${{ secrets.GITHUB_TOKEN }}
    firebaseServiceAccount: ${{ secrets.FIREBASE_SERVICE_ACCOUNT }}
    projectId: your-project-id
```

デプロイ用のサービスアカウントは、Hosting・Firestore/Storage ルールのデプロイだけができる最小権限で発行することを強く推奨します。過剰な権限を持つキーが GitHub Secrets から漏洩した場合のリスクを最小化するためです。

第 11 章 ソーシャル機能とバイラリティの設計

本章の位置づけ： いいね・コメントなどのソーシャル機能は、コア MVP のリリース後に効果を発揮する「グロース施策」です。最初の週末でここまで手を広げると、リリース自体が月曜朝に間に合わなくなるリスクが高いため、意図的にスコープ外としています。リリース後、実際のユーザーの反応を見ながら着手してください。

11-1. いいね・コメント機能

Web と iOS の両方から同じ `sharedTrips/{tripId}/likes・/comments` サブコレクションを参照・更新することで、「アプリで付けたいいねが Web にも反映される」「Web で書いたコメントがアプリにも表示される」という双方向の連動を実現します。

```
// web/src/lib/useTripLikes.ts (概念コード)
export function useTripLikes(tripId: string) {
  const [count, setCount] = useState(0);
  const [hasLiked, setHasLiked] = useState(false);
  useEffect(() => {
    const likesRef = collection(db, "sharedTrips", tripId, "likes");
    return onSnapshot(likesRef, (snapshot) => {
      setCount(snapshot.size);
      setHasLiked(snapshot.docs.some((d) => d.id === auth.currentUser?.uid));
    });
  }, [tripId]);
  const toggleLike = async () => {
    if (!auth.currentUser) {
      return;
    }
    const likeDocRef = doc(db, "sharedTrips", tripId, "likes", auth.currentUser.uid);
    if (hasLiked) {
      await deleteDoc(likeDocRef);
    } else {
```

```

    await setDoc(likeDocRef, { createdAt: serverTimestamp() });
  }
};
return { count, hasLiked, toggleLike };
}

```

11-2. なぜソーシャル機能が収益化に直結するのか

いいね・コメントといった機能は、単なる「賑やかし」ではありません。次のような形でグロース（成長）に寄与します。

- **エンゲージメントの可視化**：他人の記録に反応が付いていると、自分も記録を残したい・公開したいという動機になる
- **UGC（ユーザー生成コンテンツ）の蓄積**：ユーザーが投稿した写真・物語・感想が、開発者が用意しなくてもコンテンツとして積み上がっていく
- **シェアの起点**：「いいねが付いた」という通知や実績が、SNS でのシェアのきっかけになる

11-3. 共有しやすい導線を作る

古地図を歩いた記録を、SNS にシェアしたくなるようなビジュアル（軌跡の描画、獲得した御朱印の一覧、AI が生成した物語の一節）としてまとめて出力できる「シェア画像生成」機能は、実装コストの割に広告効果が高い投資です。

```

func renderShareImage(route: WalkRoute, mapSnapshot: UIImage) -> UIImage {
    let renderer = UIGraphicsImageRenderer(size: CGSize(width: 1080, height: 1080))
    return renderer.image { context in
        mapSnapshot.draw(in: CGRect(x: 0, y: 0, width: 1080, height: 1080))
        let text = "歩いた距離: ¥(route.distanceText) / 獲得した御朱印: ¥(route.stampCount)"
        text.draw(at: CGPoint(x: 40, y: 980), withAttributes: [.font: UIFont.boldSystemFont(ofSize: 32), .foregroundColor:
UIColor.white])
    }
}

```

このような機能は「開発者が広告費を払わなくても、ユーザーが勝手に広告塔になってくれる」仕組みそのものです。第 II 部のマーケティング編で、この考え方をさらに掘り下げます。

第 12 章 テスト・デバッグ・リリース準備（日曜夜～月曜朝）

12-1. テストすべき優先順位

個人開発ですべての機能に自動テストを書く時間はありません。優先順位としては次の順番です。

1. **お金・データ消失に関わる箇所**（課金処理、クラウド同期の書き込みロジック）
2. **位置情報の座標変換・距離計算**（バグが起きても気づきにくくコアゲームプレイに直結する）
3. **UI ロジックの分岐が多い箇所**（サインイン状態・オフライン状態・空データの表示分岐）

12-2. TestFlight による実地テスト

シミュレータでは検出できない不具合（GPS の実際の揺れ、バックグラウンド遷移、Watch との実機連携）を洗い出すには、TestFlight での実地テストが欠かせません。友人・家族に「実際に外を歩いてみてもらう」ことで、机上では気づけない体験の粗さが見つかります。

また、Web 版の Google サインインをきっかけに、TestFlight の外部テスター招待メールを Cloud Functions 経由で自動送信する仕組みを作っておくと、「Web 公開ページを見た人が、そのままテスターとして誘導される」という導線を作れます（第 2 週末以降の拡張施策。詳細は第 2-5 章「Google サインイン時の TestFlight 自動招待」参照）。App Store Connect API キー（Issuer ID・Key ID・秘密鍵）を発行し、Firebase Functions のシークレットとして安全に保管します。

日曜の夜、この週末のクライマックスにあたる作業が実際にビルドを外部テスターへ届けるプロセスです。ここでつまずくと月曜朝の審査提出が遅れるため、手順を具体的に把握しておきます。

ステップ 1：Xcode でアーカイブしてアップロードする

1. Xcode のスキームを「Any iOS Device (arm64)」に切り替える（シミュレータ向けのビルドはアーカイブできない）
2. **Product** → **Archive** を実行する
3. ビルド完了後に自動で開く Organizer（アーカイブ一覧）で、対象のアーカイブを選択し **Distribute App** → **App Store Connect** → **Upload** と進める
4. 署名は「Automatically manage signing」を選び、Xcode に任せるのが最も手数が少ない
5. アップロードが完了すると、App Store Connect 側で「処理中」の状態になる（HealthKit などのエンティトルメントを含む場合、処理に 10 分～1 時間程度かかることがある）

ステップ 2：外部テストに必須のメタデータを、ビルドが処理されるのを待つ間に埋めておく

ビルドの処理には待ち時間があるため、その間に次の情報を先に入力しておくとうまくいきます。外部テストでは、これらが未入力のままだとビルドをグループに追加できない、または審査で差

し戻されます。

- **TestFlight タブの「テスト情報 (Test Information)」**：フィードバック用メールアドレス、**プライバシーポリシーURL (外部テストでは必須)**、ベータ版の説明文
- **ビルド詳細画面の「輸出コンプライアンス (Export Compliance)」**：暗号化の使用有無に関する質問への回答

ステップ 3：ビルドを外部テストグループに追加し、ベータ版 App Review へ提出する

1. TestFlight 画面で対象の外部テストグループ（例：社外の協力者向けグループ）を開く
2. 「ビルド」欄の追加ボタンから、処理が完了したビルドを選択し、追加する
3. **外部テストグループへ初めてビルドを追加すると、その時点で自動的にベータ版 App Review へ提出される**(内部テストとは異なり、外部テストは審査を経ないと配布できない)
4. ステータスは Waiting for Review → In Review → Ready to Test（または Rejected）と遷移する。多くの場合は数時間～24 時間程度で結果が出るが、時間を保証するものではないため、この作業は日曜の夜のうちに済ませ、審査結果を待つ間に他の作業（スクリーンショット撮影、リリースノート執筆）を進めるのが効率的
5. 承認されると、外部テスターに招待通知が届き、TestFlight アプリ経由でインストール可能になる

内部テストとの違いを理解しておく：App Store Connect の「ユーザとアクセス」に登録済みのアカウント（開発チームのメンバー）を対象とする「内部テスト」は、ベータ版 App Review が不要で、ビルドをアップロードすればほぼ即座にテストできます。まずは内部テストで動作確認を済ませてから、外部テストグループに同じビルドを追加する、という順序にすると、審査待ちで手が止まる時間を最小化できます。

ベータ版 App Review で差し戻されやすいポイント：正式審査（App Store Review）ほど厳格ではありませんが、次の点は必ずチェックしてください。

- Info.plist の使用目的説明文（位置情報・写真ライブラリ・HealthKit など）の記載漏れがないか（第 9 章・第 9-4 節、および 12-3 で詳述するプライバシー説明文の不足は、正式審査だけでなくベータ版 App Review でも指摘対象になり得ます）
- アプリがクラッシュせず起動し、ログイン・主要機能への導線が実際に機能するか
- 明らかに未完成な画面（プレースホルダーのままの文言、ダミー画像）が残っていないか

12-3. App Store 申請時の注意点

App Store 審査で個人開発者がつまづきやすいポイントをまとめます。

- **プライバシー説明文の不足**：位置情報、写真ライブラリ、HealthKit など、使用する権限すべてに対応する Usage Description を Info.plist に用意する。読み取りのみの利用でも、

HealthKit のように Update の説明文まで要求される API がある点に注意（第 9 章参照）。

- **App Privacy（プライバシー「栄養成分表示」）の申告漏れ**：App Store Connect 上で、収集するデータの種類（位置情報、写真、識別子など）を正確に申告する。
- **外部 AI サービスの利用に関する説明**：AI が生成したコンテンツを含む場合、その旨をアプリ内・審査担当者向けのメモに明記しておくトスムーズ。
- **サインイン方法の代替手段**：Google サインインのみを提供する場合、Sign in with Apple の必須要件に抵触しないか確認する（外部アカウントサービスでのサインインを提供する場合、Apple は原則 Sign in with Apple の提供も求めるが、条件によって免除されるケースもあるため、最新のガイドラインを確認すること）。

初回リリース時のバージョンノートは、機能の羅列ではなく「このアプリで何が体験できるか」を一文で伝えることを意識します。

「江戸の古地図を持って、今日の散歩をタイムトラベルに変えよう。」

このような一文は、後述する App Store のプロダクトページやスクリーンショットのキャッチコピーにも転用できます。

12-4. 月曜早朝：リリース完了の最終チェックリスト

第 2 章で定義した「リリース完了」の状態に到達したかどうかを、月曜の朝、次のチェックリストで確認します。すべてにチェックが付けば、この週末のゴールは達成です。

- ☐ Web 公開ページが本番 URL（独自ドメインまたは Firebase Hosting 既定 URL）で実際にアクセスでき、古地図オーバーレイの見た目・サンプル記録が表示される
- ☐ Web 版の Firestore セキュリティルールが本番用に正しくデプロイされている（開発中の緩いルールのまま公開していないか再確認する）
- ☐ iOS アプリのビルドが App Store Connect にアップロード済みで、内部テストで実機動作を確認済みである
- ☐ TestFlight の外部テストグループにビルドが追加され、ベータ版 App Review に提出（または承認）されている
- ☐ App Store Connect 上で、アプリ名・サブタイトル・スクリーンショット・プライバシー情報（App Privacy）・年齢区分が入力済みである
- ☐ Submit for Review（審査へ提出）ボタンを実際に押し、ステータスが「審査待ち（Waiting for Review）」になっている
- ☐ SNS 告知用の投稿文・画像を用意し、審査通過後すぐに公開できる状態にしてある

審査提出後の承認・一般公開までには通常数時間～数日を要しますが、「**金曜夜に始めて、月曜朝ま**

でに審査へ提出し終えている」という状態そのものが、この週末のゴールです。審査結果を待つ間は、次の週末で拡張する機能（第 9 章の Watch 対応、第 11 章のソーシャル機能）の計画や、第 II 部で扱うマーケティング準備を進めてください。

第 II 部 収益化・マーケティング編

第 13 章 地図ゲームアプリの収益化モデルを選ぶ

13-1. 収益化の 4 つの型

個人開発アプリの収益化モデルは、大きく 4 つに分類できます。

モデル	特徴	地図ゲームとの相性
買い切り課金	初回に一括で支払ってもらう	ダウンロード数が伸びにくい。ニッチな古地図テーマでは母数不足になりがち
アプリ内課金（消費型）	追加の古地図パック、AI 生成回数の追加購入など	相性が良い。コンテンツを「地域パック」として売れる
サブスクリプション	月額・年額で全古地図・無制限 AI 生成を提供	継続利用が前提のウォーキングゲームと非常に相性が良い
広告	バナー・インタースティシャル・リワード広告	単体では収益性が低いですが、無料ユーザーの母数を活かす補完的手段として有効

13-2. おすすめの組み合わせ：フリーミアム + 地域パック課金

本書で推奨するのは、次のようなハイブリッドモデルです。

- **無料で使える範囲**：1～2 エリアの古地図、基本的な記録・チェックポイント機能
- **アプリ内課金（消費型・非消耗型）**：地域ごとの古地図パック（例：「本郷・谷中エリアパック」「東海道パック」）を 1 つずつ購入できるようにする
- **サブスクリプション（任意）**：全エリア古地図の解放 + AI 生成物語を無制限に使えるプランを月額で提供
- **広告（任意・補完的）**：無料ユーザー向けに、チェックポイント到達時のリワード広告（視聴すると追加ポイント）を選択制で提供

このモデルの利点は、「無料で試して、気に入ったエリアだけ課金する」という体験が、地図ゲームというジャンルの性質（地域性・観光性）と自然に噛み合う点です。全国一律のサブスクリプションだけを提示するよりも、「自分の住んでいる街、旅行に行く街のパックだけ買う」という選択肢の方が、心理的なハードルが低くなります。

13-3. StoreKit でのアプリ内課金実装の要点

import StoreKit
@MainActor

```

final class StoreManager: ObservableObject {
    @Published var products: [Product] = []
    @Published var purchasedProductIDs: Set<String> = []
    func loadProducts(ids: [String]) async {
        products = (try? await Product.products(for: ids)) ?? []
    }
    func purchase(_ product: Product) async {
        guard let result = try? await product.purchase() else { return }
        switch result {
        case .success(let verification):
            if case .verified(let transaction) = verification {
                purchasedProductIDs.insert(transaction.productID)
                await transaction.finish()
            }
        default:
            break
        }
    }
}

```

課金対象のプロダクト ID は、App Store Connect 側で「地域パック」ごとに設定し、購入済み ID のセットに応じて OldMapCatalog のロック状態を制御します。

13-4. 「無料枠」の設計は慎重に

無料枠を広げすぎると課金の動機がなくなり、狭めすぎると初回の体験価値が伝わらずアンインストールされます。目安として、「無料ユーザーが最初のセッションで“これは面白い”と感じられるだけの古地図・チェックポイント数（3～5 箇所程度）」を無料開放し、それ以上の物量を課金対象にする設計から始め、実際の利用データ（後述の分析）を見ながら調整するのが現実的です。

第 14 章 ASO（App Store 最適化）の基本

14-1. アプリ名・サブタイトルの設計

App Store での検索流入は、広告費をかけない個人開発者にとって最も重要な集客チャネルです。アプリ名とサブタイトルには、検索されるであろうキーワードを自然な形で含めます。

- アプリ名：ブランド名 + 差別化ワード（例：「Komap - 古地図で街歩き」）
- サブタイトル：機能や体験を端的に（例：「古地図 × GPS ウォーキング記録」）

「地図」「散歩」「ウォーキング」「観光」「歴史」「御朱印」「聖地巡礼」など、実際にユーザーが検索しそうな語彙を洗い出し、App Store Connect の「キーワード」欄（100 バイトまで）に、アプリ名・サブタイトルと重複しない形で詰め込みます。

14-2. スクリーンショット、動画は「体験」を見せる

スクリーンショットは機能一覧ではなく「使っているとどんな気持ちになるか」を伝えるべきです。

1. 1 枚目：一番魅力的な体験の瞬間（古地図が重なった地図のビジュアル）＋キャッチコピー
2. 2 枚目：チェックポイントに近づいて御朱印を獲得する瞬間
3. 3 枚目：AI が生成した物語を読んでいる画面
4. 4 枚目：歩いた軌跡が地図上に描かれた達成感のある画面
5. 5 枚目：Web 版や Apple Watch 連携など、エコシステムの広さを示す画面

15～30 秒のプレビュー動画は、テキストや静止画よりコンバージョン率を高める傾向があります。実際に外を歩いて記録している様子、透過度スライダーを操作して古地図が浮かび上がる瞬間など、「動きがあるからこそ伝わる」体験を優先的に収録します。

- アプリ内の適切なタイミング（記録を保存し終えた直後、御朱印を複数獲得した直後など、ポジティブな体験の直後）で SKStoreReviewController によるレビュー依頼を表示する
- ネガティブなレビューには、可能な範囲で開発者からの返信を行い、改善につなげている姿勢を見せる（新規ユーザーはレビューへの返信も判断材料にする）

第 15 章 低予算マーケティング戦略

15-1. 個人開発者が使える広報チャネル

大きな広告予算を持たない個人開発者でも次のようなチャネルは無料/低コストで活用できます。

- **SNS (X/Instagram/TikTok)**：実際に古地図を持って街を歩く様子を動画で発信する。「Before/After」（現在の風景⇄古地図の重ね合わせ）は視覚的にバズりやすいフォーマット
- **note・ブログでの開発記録公開**：「個人開発でここまで作った」というプロセス自体がコンテンツになる（本書のような開発記もその一例）
- **地域メディア・観光協会への持ち込み**：古地図・郷土史というテーマは、地方紙やローカルテレビ、観光協会の SNS と相性が良く、取材してもらえる可能性がある
- **Product Hunt 個人開発者コミュニティでの公開**：エンジニアアーリーアダプター層への露出
- **プレスリリース配信サービス**：無料～低価格のプレスリリース配信サービスを使い、「〇〇市の古地図アプリ」のような地域性のあるニュースとして配信する

15-2. 「Before/After」コンテンツの作り方

古地図オーバーレイの透過度を 0%（現在の地図）から 100%（古地図のみ）へスライドさせる様子を録画するだけで、視覚的にインパクトのあるショート動画が作れます。これはアプリの核となる機能そのものを、追加の演出なしでマーケティングコンテンツに変換できる、非常にコストパフォーマンスの高い手法です。

```
func playOverlayDemo() {
  withAnimation(.easeInOut(duration: 3)) {
    overlayOpacity = overlayOpacity > 0.5 ? 0.0 : 1.0
  }
}
```

15-3. コミュニティ・ローカルパートナーシップ

- 地域の史跡ガイドボランティア団体、郷土資料館、観光案内所などに、アプリの存在を伝える（無料掲載や協力を打診できる可能性がある）
- 学校の郷土学習・総合学習の教材としての活用を提案する（教育目的の利用は、収益に直結しなくても口コミの起点になる）
- ウォーキングイベント、スタンプラリーイベントの主催者に、アプリを「デジタルスタンプラリー」として提供する提案をする

15-4. ローンチ時の「初速」を作る施策

App Store のランキングアルゴリズムは、短期間でのダウンロード数・エンゲージメントの伸びを重視する傾向があります。個人開発者でも次のような工夫で初速を作れます。

- リリース日を事前に告知し、SNS のフォロワーにリリース当日にダウンロードを呼びかけ
- ローンチ記念として、期間限定で全古地図パックを無料開放する
- TestFlight で先行体験したテスターに、正式リリース日にレビュー投稿を依頼する（自然な形での複数レビューの獲得）

第 16 章 グロースサイクルと継続的改善

16-1. 計測すべき指標

収益化・マーケティングを継続的に改善するには、次の指標を最低限追う体制を作ります。

- **DAU/MAU（デイリー／マンスリーアクティブユーザー）**：継続利用の指標
- **リテンション率（1 日後・7 日後・30 日後）**：初回体験の質、ゲーミフィケーションの効果
- **課金率（無料ユーザーのうち課金に至った割合）と ARPU（ユーザーあたり平均収益）**
- **チェックポイント到達率、古地図ごとの利用比率**：どのコンテンツが人気か把握し、次に作る古地図の優先順位付けに使う

Firebase Analytics を導入すれば、これらの指標の多くを追加の開発コストをほとんどかけずに計測できます。

```
import FirebaseAnalytics
Analytics.logEvent("checkpoint_reached", parameters: [
    "map_id": site.mapId,
    "site_id": site.id,
])
```

16-2. リテンションを上げる小さな改善の積み重ね

本書のロードマップで紹介した「マイ時空旅の一覧」「直近3件だけ表示して折りたたむ」「日付表記をM/D形式に短縮する」といった細かなUI改善は、単体では地味に見えますが、日々の利用体験の摩擦を減らし、結果的にリテンションに寄与します。個人開発では大きな新機能よりも、こうした小さな改善のサイクルを高速に回せることが強みになります。

16-3. コンテンツの継続的な追加

地図ゲームアプリのコンテンツ（古地図・史跡チェックポイント）は、一度作って終わりではなく、継続的に追加していくことでユーザーの「まだ知らない場所がある」という好奇心を維持できます。

- 季節ごとのイベント（花見シーズンに合わせた古地図の追加、夏祭にちなんだ史跡の追加）
- ユーザーからのリクエストを募る仕組み（「次にどのエリアの古地図が欲しいですか？」というアンケート機能）
- ユーザー自身が史跡候補を投稿できる仕組み（UGC化によるコンテンツのスケール）

16-4. 価格・プラン改定のテスト

App Store Connect の価格帯変更や、サブスクリプションのイントロ価格（初回割引）機能を使い、小規模なA/Bテストに近い形で価格施策を試すことができます。個人開発では厳密なA/Bテスト基盤を持つことは難しいため、「一定期間ごとに価格・訴求文言を変えて、コンバージョン率の変化を観察する」という簡易的なPDCAで十分です。

第17章 ケーススタディ：週末開発から一つのアプリができるまで

本章では、これまでの章で紹介した技術・考え方を、実際に1つのアプリとして組み立てていった記録を時系列でたどります（本書執筆にあたって参照した、実在のプロジェクトの開発履歴に基づく再構成です）。

なお、本章で振り返るプロジェクトの実際の歴史は、第2章で提示した「1つの週末で作ってリリースする」という型が確立される前に、複数回の週末を重ねて少しずつ育てられたものです。そのため本章の前半（17-1～17-2）は、第2章のコアMVPに相当する範囲を、当時は数回の週末に分けて実装した記録として読んでください。読者が本書のフローに沿って新しく開発する場合は、17-1～17-2の内容は「最初の1つの週末」に圧縮され、17-3以降の内容が「2つめ以降の週末」で実現する拡張フェーズに相当します。

17-1. 最小限のプロトタイプ期

最初の数週末は、Google Maps SDK の導入、現在地表示、1 枚の古地図オーバーレイの表示に費やされました。この段階では UI も粗く、古地図の位置合わせも大まかな仮の値でしたが、「古地図が現在の地図に重なって見える」という核となる体験は、この時点ですでに成立していました。

17-2. コア体験の拡張期

古地図のカタログ化、チェックポイントと御朱印の仕組み、GPS による徒歩ルートの記録・保存が実装され、「歩く」という行為そのものがゲームになっていきました。この段階で、テキストチャートラスの制約や、SwiftUI の Menu の表示件数問題など、SDK・フレームワーク特有の制約に何度も遭遇し、その都度「表示件数の制限」「画像の事前縮小」「UI コンポーネントの構成変更」といった実務的な回避策で対処していきました。

17-3. AI とクラウドによる体験の深化期

OpenAI API による物語生成、Firebase によるクラウド同期、Firebase Storage を使った写真投稿機能が加わり、単なる記録アプリから「その場所の物語に出会えるアプリ」へと体験が深まりました。この段階でセキュリティルールの設計、API キーの安全な管理など、個人開発者が見落としがちなセキュリティ面の作業が発生しています。

17-4. エコシステム拡張期

Apple Watch 単体アプリ、Web アプリ版が追加され、「iPhone がなくても記録できる」「ブラウザからも記録を見返せる」という形でエコシステムが広がりました。この段階で、Watch 単体記録の連動状態の整合性、Web 版での自分の記録の編集機能、いいね・コメントといったソーシャル機能が実装され、単一デバイスのアプリから「複数の接点を持つサービス」へと進化しています。

リリース後も、「マイ時空旅の一覧の折りたたみ表示」「日付表記の短縮」「プレビュー地図の軽量化」「回転が必要な古地図の向きのずれ修正」といった細かな改善が継続的に行われています。これらは派手な新機能ではありませんが、実際に使ってみて初めて気づく体験の粗さを一つずつ潰していく作業であり、個人開発における「磨き込み」の重要性を象徴しています。

17-5. このケーススタディから学べること

- 完璧な設計を最初から目指さず、「仮の値」「大まかな実装」で動くものを先に作る
- SDK・フレームワークの制約は、ドキュメントよりも実際に手を動かして発見することが多い。遭遇したら記録し、再発防止の仕組み（縮小処理の共通化、UI コンポーネントの置き換えなど）に落とし込む
- 機能追加の合間に、細かな UI 改善・不具合修正を挟み込むことで、リリース後も「育て続けているアプリ」であることをユーザーに伝えられる
- 複数回の週末に分けて育てる場合でも、常に「今回の週末で、どこまでを“動く状態で終える”か」というスコープの区切りを事前に決めておく——という規律そのものは、第 2 章で提

示した「1つの週末で作り切る」モデルにも、複数回に分けて育てるモデルにも共通する、開発を止めないための唯一の原則です

第18章 法務・プライバシー・倫理面の留意点

18-1. 位置情報の取り扱い

位置情報は個人を特定しうるセンシティブなデータです。次の点に留意してください。

- プライバシーポリシーに、収集する位置情報の種類・利用目的・第三者提供の有無を明記する
- 必要最小限の精度・頻度でのみ位置情報を取得する（常時バックグラウンドで高精度取得を行う必要が本当にあるか、都度見直す）
- クラウドに保存する位置情報（軌跡データ）は、本人以外がアクセスできない設計を徹底する（第8章のセキュリティルール参照）

18-2. 著作権・史料の扱い

古地図や歴史資料を使う場合、著作権の状況を必ず確認してください。

- パブリックドメインとなっている歴史地図（発行から一定年数が経過し著作権が消滅しているもの）は、出典を明記した上で利用可能な場合がありますが、配布元・アーカイブのライセンス条件も別途確認する必要があります
- 現在の地図画像（Google Maps、OpenStreetMap など）を加工して「古地図風」に仕立てる場合も、各サービスの利用規約・ライセンス（例：OpenStreetMap の ODbL ライセンスでは帰属表示が必要）を遵守してください
- 実在の史跡・地名を扱う場合、地域住民や関係団体への配慮（誤った歴史情報を断定的に発信しない、宗教施設への配慮など）も忘れずに

18-3. AI 生成コンテンツの取り扱い

AI が生成する物語やエピソードには、事実と異なる内容が含まれる可能性があります。アプリ内に「AI による生成コンテンツを含みます。史実と異なる場合があります」といった注意書きを設けることは、ユーザーへの誠実さであると同時に、後のトラブル回避にもつながります。

18-4. 未成年ユーザーへの配慮

位置情報や写真投稿機能を含むアプリは、未成年ユーザーが利用する可能性も考慮する必要があります。App Store の年齢区分設定を適切に行い、必要に応じて保護者の同意フローや、投稿内容のモデレーション（不適切な投稿の通報・削除機能）を検討してください。

第 19 章 よくある質問 (FAQ)

Q. プログラミング未経験でも本書の内容を実践できますか？

A. Swift・SwiftUI の基礎的な文法（変数、関数、クラス、SwiftUI の View 構文）を理解していることを前提としています。未経験の場合は、まず SwiftUI の入門教材で基礎を固めてから本書に取り組むことをお勧めします。一方、Web 版やマーケティング編は、非エンジニアの方が「外注する際に何を発注すればよいか」を理解する目的で読むこともできます。

Q. Google Maps SDK は無料で使えますか？

A. Google Maps Platform には無料利用枠があります（利用状況やタイミングによって条件は変わるため、必ず Google Cloud Platform の最新の料金ページを確認してください）。個人開発の検証段階であれば無料枠内に収まることが多いですが、リリース後にユーザー数が増えた場合の請求額の上限設定（予算アラート）を必ず行っておきましょう。

Q. Firebase の無料枠だけで運用できますか？

A. Firestore の読み書き回数、Cloud Functions の実行回数、Storage の容量にはいずれも無料枠（Spark プラン）がありますが、Cloud Functions を使った外部 API 呼び出し（メール送信など）には Blaze プラン（従量課金）への切り替えが必要です。ユーザー数が少ない初期段階では無料枠内に収まることが多いですが、予算アラートの設定は必須です。

Q. 古地図の著作権が心配です。オリジナル画像で代替できますか？

A. 可能です。本書で紹介した「現在の地図をセピア調に加工して古地図”風”に仕立てる」手法や、AI 画像生成サービスでイラスト風の地図を作る手法であれば、著作権の懸念を大きく減らせます。ただし、加工元となる地図タイル画像自体の利用規約（帰属表示の要否など）は確認してください。

Q. 収益化まで、実際どれくらいの期間がかかりますか？

A. 開発ボリューム・マーケティング施策の実行度合いによって大きく異なります。1 つの週末でコア MVP をリリースした場合でも、そこから収益化モデルの導入とマーケティング施策の実行を経て初めての売上が立つまでには、さらに数週末～数ヶ月を見込むのが現実的です。焦らず、まずは最初の週末で自分が使いたいと思えるアプリを世に出し切ることを優先してください。

Q. 本当に 1 つの週末（金曜夜～月曜朝）だけでリリースまで終わりますか？無理があるのでは？

A. 正直に言えば、初めて Google Maps SDK や Firebase に触れる方にとっては、かなりタイトなスケジュールです。第 2 章で「コア MVP の範囲をとことん絞り込む」ことを強調しているのは、まさにこのためです。もし土曜の夜の時点で明らかに遅れている場合は、無理に全ブロックを詰め込まず、次の 2 つの調整のどちらかを取ってください。

1. **Web 版の本番公開だけをこの週末のゴールとし、iOS 版の App Store 提出は次の週末に持ち越す**（Web 版は審査がなく即座に公開できるため、「何かを世に出した」という達成感を

最小限守れます)

2. **チェックポイントの数や古地図のエリアをさらに削り、“歩いて1つだけ御朱印が獲得できる”という最小の垂直スライスだけをリリースする**

いずれの場合も、「完成度を下げてでも、時間内に何かを世に出す」という優先順位を貫くことが、次の週末につながるモチベーションを守ります。

Q. ベータ版 App Review、または App Store 本審査で却下 (Rejected) されたら、この週末の計画はどうなりますか？

A. 却下は個人開発では珍しいことではなく、多くの場合、審査担当者からの却下理由 (Resolution Center 内のメッセージ) を読めば対応方法が明確に分かります。第 12 章で触れたプライバシー説明文の不足や、App Privacy の申告漏れは、却下理由として特によく見られるものです。却下された場合の現実的な対応は、次の 2 段階です。

1. **却下理由に対応する修正を行い、同じ週末中、あるいは平日の隙間時間に再提出する** (多くの却下理由は数十分～数時間で修正できる軽微なものです)
2. **どうしても週末中に解決できない場合は、Web 公開ページの公開というゴールだけは死守し、iOS 版の再提出を翌週の作業に持ち越す**

審査却下を過度に恐れて、事前の作り込みに時間をかけすぎないようにしてください。個人開発における審査対応は「一発で通す」ことよりも「却下されても素早く直して出し直す」姿勢の方が、結果的に速くリリースにたどり着けます。

第 20 章 3 人の起業家哲学から学ぶ収益化・マーケティング戦略

これまでの章では、収益化モデルやマーケティング施策を機能・手法の単位で解説してきました。本章では視点を換え、プロダクト哲学の異なる 3 人の起業家——**スティーブ・ジョブス**（Apple 共同創業者）、**ピーター・ティール**（PayPal 共同創業者、投資家、『ゼロ・トゥ・ワン』著者）、**イーロン・マスク**（Tesla・SpaceX 創業者）——それぞれの思考様式を「フレームワーク」として取り出し、地図ゲームアプリの「無料／有料の区分け」「プロダクトとしての魅力の作り方」「ユーザーの見立て（誰に向けて作るか）」に当てはめて考えます。

3 人はそれぞれ全く異なる経営哲学を持ちますが、共通しているのは「中途半端を嫌う」という一点です。本章を読んだ後、実際にどう手を動かすかは第 20-4 節の「Komap 適用チェックリスト」にまとめています。

20-1. スティーブ・ジョブス型：「体験の完成度」で無料化を拒否する

哲学の核

ジョブスの哲学は一言で言えば「プロダクトそのものが広告であり、プロダクトそのものが価格である」というものです。ジョブスは値引き・無料化によって普及を図ることを嫌い、代わりに「これしかない」と思わせる体験の完成度そのものに投資しました。iPhone には常に「and これもできます」という機能列挙ではなく、基調講演で語られる「たった 1 つの物語」がありました。

ジョブス哲学を分解すると、次の 4 つの原則になります。

1. **フォーカス（No, No, No, and then Focus）**：やらないことを決める。機能を足すのではなく、削ることで際立たせる。
2. **エンド・ツー・エンドの体験設計**：ハードウェア・ソフトウェア・パッケージング・店頭体験まで、すべてを一つの世界観で統一する。
3. **プレミアム・ポジショニング**：安売りをしない。値引きは「この商品には値引きしてでも売る理由がある」というメッセージを市場に送ってしまうため、避ける。
4. **ストーリーテリング・マーケティング**：スペック表ではなく物語（Before/After の感情の動き）で語る。基調講演のような「体験のデモ」を中心に据える。

無料／有料の区分け方（ジョブス型）

ジョブス型のアプローチでは、「**無料版は“完成度の低い試供品”ではなく、“完全に磨き込まれた小さな体験”にする**」という発想を取ります。無料枠を機能制限で作るのではなく、「体験としてどこまでも完璧な、しかし対象範囲が狭いもの」として設計します。

- 無料版：1 エリア（例えば自宅周辺の徒歩 10 分圏）だけを、演出・アニメーション・AI 生成の物語のクオリティを一切妥協せず、完全な形で提供する

- 有料版：エリアを追加していく「地域パック」という形にするが、パックそのものの体験の質は無料版と全く同じレベルを維持する（「有料だから雑」を絶対に作らない）
- サブスクリプションはあえて前面に出さず、「これは体験を買うものだ」という文脈を保つために、価格表示よりも先に体験のデモ（動画・スクリーンショット）を見せる

プロダクトとしての魅力の作り方（ジョブズ型）

- **1つの強い機能に絞ったキャッチコピー**を作る（例：「地図が、タイムマシンになる。」）。複数の機能を並列で語らない。
- **オンボーディング（初回起動体験）を演出として設計する**：チュートリアルではなく、「最初の古地図が浮かび上がる瞬間」を1つの感動的な体験として作り込む。Apple 製品のパッケージを開ける体験（アンボクシング）に相当する、アプリ版の「儀式」を用意する。
- **UI から機能を削る**：設定項目、選択肢を極力減らす。「古地図を選ぶ」という行為すら、賢いデフォルト（現在地から最も近い、最も評価の高い古地図を自動選択）によって省略できないか検討する。

ユーザーの見立て方（ジョブズ型）

ジョブズは「顧客は自分が何を欲しいか分かっていない」と考え、マーケットリサーチよりも自分自身の審美眼を信じました。地図ゲームに当てはめると、「アンケートを取って機能を定める」のではなく、「開発者自身が“これは美しい、感動する”と確信できる古地図・体験だけを選び抜いて世に出す」という姿勢になります。ユーザーセグメントを細かく分けるのではなく、「**審美眼の合う少数の熱狂的なファン**」に照準を絞り、**その人たちが友人に見せたいというプロダクトを作る**ことを優先します。

20-2. ピーター・ティール型：「独占できるニッチ」から始めて拡大する

哲学の核

ティールの哲学の核心は『ゼロ・トゥ・ワン』に集約される「競争は負け犬のするものだ（Competition is for losers）」という主張です。競争の激しい市場でシェアを奪い合うのではなく、**最初は小さくても独占できるニッチ市場を見つけ、そこを完全に支配してから隣接市場へ同心円状に拡大する**という戦略を取ります。有名な例が、Facebook が最初「ハーバード大学の学生専用」という極小市場から始めたことです。

ティールはスタートアップが成功するために答えるべき7つの質問を提示しています。地図ゲームに当てはめると次のようになります。

ティールの問い 地図ゲームへの適用

エンジニアリング：10 倍 古地図の位置合わせ精度・AI 生成の質・UX の滑らかさで、既存の観光アプリの 10 倍の体験を作れているか

ティールの問い 地図ゲームへの適用

タイミング：今始めるべき理由があるか 生成 AI コストの低下、GPS 精度の向上、地域観光需要の高まりという今のタイミングを活かしているか

独占：小さな市場を独占できるか 「全国の古地図」ではなく、まず「特定の 1 つの街の古地図」市場を完全に独占できているか

人材：正しいチームか 個人開発でも、地図・史料・AI・マーケティングの役割を自分がどう兼務し、どこを外部委託するかが明確か

販売・流通：製品を届け方法があるか ASO・SNS・地域メディアなど、実際に使える流通チャネルを具体的に持っているか

持続性：10 年後も生き残る優位性があるか 蓄積されたユーザーの記録データ、コミュニティ、独自の古地図アーカイブなど、模倣されにくい資産を積み上げているか

秘密：他人が気づいていない真実を知っているか 「地図アプリは無料で当然」という業界通念に対し、「地域密着コンテンツになら喜んで課金する層がいる」という自分だけの仮説を持っているか

無料／有料の分け方（ティール型）

ティール型のアプローチは、「**無料は独占するニッチ市場を作るための“参入障壁の構築”に使い、有料は独占した市場からの価値の回収に使う**」という考え方です。

- 無料版：最初に独占を狙う「1 つの街・1 つのテーマ」（例：地元の商店街の古地図、特定の観光地の史跡ラリー）を無料で徹底的に作り込み、その街では代替のきかない「デファクトスタンダード」になることを目指す
- 有料版：独占した市場の中で、地域の観光協会・自治体・商店会と提携した「公式スタンプラリー機能」「企業タイアップ古地図パック」など、競合が簡単には模倣できない独自コンテンツを有料で提供する
- 価格は「価値創出（Value Creation）」ではなく「価値の獲得（Value Capture）」で決める。無料の観光アプリが乱立する中でも、「このアプリでしか手に入らない古地図・スタンプ」には競争が及ばないため、価格競争に巻き込まれない

プロダクトとしての魅力の作り方（ティール型）

- **ネットワーク効果を組み込む**：ユーザーが増えるほど価値が増す仕組み（他ユーザーの投稿写真・コメント・いいねが蓄積されるほど、その街の古地図コンテンツが充実して見える）を意図的に設計する
- **独自資産（プロプライエタリな技術・データ）を蓄積する**：AI で生成した物語、ユーザーが投稿した写真・感想は、時間が経つほど競合が同じものを再現できない資産になる。これを「模倣困難性（Moat、堀）」として明確に意識する

- **べき乗則（Power Law）で考える**：すべての街・すべての機能に均等にリソースを割かず、最も成功する可能性が高い「1つの街」「1つのキラー機能」に集中投資する

ユーザーの見立て方（ティール型）

ティール型では、広く浅いユーザーベースではなく、「**特定の地域・特定の興味関心を持つ、小さいが熱量の高い集団**」を最初のターゲットに定めます。地図ゲームであれば、次のようなセグメントが候補になります。

- 特定の街の郷土史サークル・歴史愛好家コミュニティ（数百人規模でも、深く使い込んでくれる）
- 特定の観光地のリピーター層（一度訪れて気に入り、また来たいと思っている層）
- 地域の小学校・中学校の郷土学習の先生（教材として使ってもらえれば、生徒という大きな二次的ユーザー層に波及する）

「まず1つの街を完全に獲る」という発想が、ティール型の最大の実践ポイントです。

20-3. イーロン・マスク型：「ミッション」と「第一原理」でファンを動員する

哲学の核

マスクの哲学は、**ミッション（大義）の提示による熱狂的な支持者の獲得と、第一原理思考（First Principles Thinking）による大胆なコスト構造の再設計**の組み合わせです。Tesla は単なる電気自動車メーカーではなく「持続可能なエネルギーへの移行を加速する」というミッションを掲げ、そのミッションに共感するファンが広告費をかけずに製品を広めました。また「最良の部品とは、存在しない部品である（The best part is no part）」という第一原理思考で、不要な工程・機能を徹底的に削ぎ落とし、コストと価格を下げました。

マスク型のマーケティングのもう一つの特徴は、**創業者自身が製品の一番のマーケター（伝道者）になる**という点です。SNS での直接発信、技術デモの公開、批判にも臆さず反応する姿勢が、良くも悪くも強い注目とファンダムを生みます。

無料／有料の区分け方（マスク型）

マスク型のアプローチは、「**無料はネットワーク効果とデータ収集のための“くさび（Wedge）”として使い、有料は規模の経済で実現したコスト構造の優位性を価格に転嫁する**」という考え方です。

- 無料版：基本的な記録・古地図体験を無料で広く提供し、できるだけ多くのユーザーに「歩いた軌跡データ」「訪問した史跡データ」を蓄積してもらう。このデータ自体が、次の古地図・史跡コンテンツを作る際の「どこに需要があるか」を示す一次情報になる
- 紹介プログラム（リファラル）：Tesla の紹介制度に倣い、「友人を招待して一緒に歩くと、両方に追加の古地図パックが無料で解放される」といった、ユーザー自身が拡散の担い手になる仕組みを作る

- 有料版：規模が大きくなった段階で、AI による物語生成をより大規模・低コストに提供できるようになった優位性（第一原理で組み直したコスト構造）を活かし、「業界標準よりも安い」価格でサブスクリプションを提供する。値下げによる普及速度の最大化を狙う（ジョブス型のプレミアム戦略とは対照的な発想）

プロダクトとしての魅力の作り方（マスク型）

- **ミッションを明文化する**：「消えていく郷土の記憶を、歩くだけで未来に残す」といった、単なる機能紹介を超えた大義をアプリの説明文・SNS プロフィール・アプリ内メッセージに一貫して掲げる
- **開発者自身が発信者になる**：開発の裏側、古地図の発見エピソード、技術的なチャレンジ（テキストチャートラスの制約を乗り越えた話など）を、開発者個人の SNS アカウントで定期的に発信し、「作っている人の物語」自体をファンダム形成のコンテンツにする
- **大胆な公開デモ**：新しい古地図・新機能をリリースする際、SNS のライブ配信や動画で「今の場所を実際に歩きながら」機能を実演する。スペックの説明よりも、実演によって信頼と驚きを作る
- **第一原理でコストを再設計する**：AI 生成のプロンプト・モデル選定、画像処理のパイプラインなど、当たり前とされているコスト構造を疑い、「本当に必要な処理は何か」を突き詰めて再設計する。浮いたコストを価格の安さやコンテンツ量の多さに還元する

ユーザーの見立て方（マスク型）

マスク型では、ユーザーを「顧客」としてだけでなく「**ミッションに共感する仲間・伝道者**」として見立てます。ターゲットセグメントの例は次の通りです。

- SNS での発信力が高いアーリーアダプター層（彼らが拡散のハブになる）
- 環境・地域活性化・文化継承といった社会的なテーマに関心の高い層（ミッションへの共感が継続利用の動機になる）
- 「友達を誘って一緒に遊べる」ことに価値を感じるソーシャル志向のユーザー層（紹介プログラムの担い手）

20-4. 3つの哲学の使い分けと組み合わせ方

3人の哲学は対立するようでいて、実は「フェーズ」で使い分けることができます。

1. **立ち上げ期（ティール型）**：まず1つの街・1つのテーマで独占を狙う。広く浅くではなく、狭く深く。
2. **体験の磨き込み（ジョブス型）**：独占したニッチの中で、体験の完成度を極限まで高める。安売りせず、「これしかない」という満足度を作る。
3. **拡大期（マスク型）**：磨き上げた体験とミッションを掲げ、無料枠とリファラルでユーザー

ベースを一気に拡大し、規模の経済で実現した優位性を新たな価格・コンテンツ量に還元する。

個人開発者が1人で全フェーズを同時に実践するのは困難です。「**今、自分のアプリはどのフェーズにいるか**」を意識し、**そのフェーズに合った哲学を主軸に据える**ことが実践上のコツです。

20-5. Komap 適用チェックリスト

ここからは、本書のケーススタディで扱ってきた実在のアプリ「Komap」（古地図オーバーレイ×GPS ウォーキング×AI 物語生成×御朱印収集×Firebase 同期×Web/Watch 展開）に、上記3つの哲学を実際に当てはめた場合に「やるべきこと」を、フェーズ別・哲学別のチェックリストとして整理します。

フェーズ1：独占するニッチを決める（ティール型）

- Komap の実在の古地図カタログ（本郷・谷中・上野／日本橋／芝／神田／東海道／中山道など）の中から、**最初に完全制覇を狙う「1 エリア」を1つだけ選ぶ**（例：本郷・谷中・上野エリア。既に他古地図との統合実績があり、コンテンツ密度を上げやすい）
- そのエリアの郷土史サークル・地域観光協会・地元の歴史ガイドボランティア団体をリストアップし、直接連絡を取る（無料での情報提供・協業を打診）
- そのエリアの史跡チェックポイント（HistoricSite）を、現状より密度高く追加し、「このエリアなら他のどのアプリより詳しい」と言える状態を作る
- sharedTrips のいいね・コメント機能を使い、そのエリアのユーザー投稿（写真・感想）が可視化されるダッシュボードを Web 公開ページに設け、「このエリアで一番使われているアプリ」という社会的証明を作る
- 自治体・観光協会向けに、Firestore の匿名集計データ（人気の史跡・滞在時間帯など）を使った簡易レポートを無償提供し、将来の公式タイアップ（有料の「地域パック」の布石）につなげる

フェーズ2：体験を磨き込む（ジョブス型）

- 現状「江戸城周辺（安政期）」のようにイラスト画像として作られている古地図について、フェーズ1で選んだ主力エリアだけは位置合わせ精度を手作業で見直し、「仮の値」から「実測に近い値」へ精度を引き上げる
- AI が生成する物語（AIHistoryService）のプロンプトを主力エリアだけ特別にチューニングし、他のエリアより明らかに質の高い物語が出るようにする（無料の主力エリアこそ一切妥協しない）
- オンボーディング（初回起動～最初の古地図が浮かび上がる瞬間）を専用の演出としてリデザインし、「アプリが起動していきなり地図が出る」のではなく、透過度が0→100%へゆっくりアニメーションする「儀式」化した導入体験を作る

- 設定画面・古地図選択の選択肢を見直し、現在地から最も近い・最も評価の高い古地図を自動的に提案するデフォルト挙動を追加し、選択の手間を削る
- マーケティング文言（App Store 説明文・Web 公開ページの見出し）を、機能列挙から「地図が、タイムマシンになる。」のような一文のキャッチコピー中心の構成に書き換える

フェーズ3：拡大する（マスク型）

- Komap の利用規約・アプリ内メッセージ・Web 公開ページに、「消えていく郷土の記憶を、歩くだけで未来に残す」といった一貫したミッション文を明記する
- リファラル機能を新規に設計・実装する：友人を招待し、2 人が同じエリアを一緒に歩いて記録すると、双方に追加の古地図パック（または AI 生成回数の上乗せ）を無料で解放する仕組みを SyncService・Firestore に追加する
- 開発者自身（または Komap 公式アカウント）の SNS で、古地図の発見エピソードや位置合わせの苦労話、テキストチャートラス制約を乗り越えた開発の裏側などを定期的に発信し、「作っている人の物語」を継続的なコンテンツにする
- 新しい古地図・新機能をリリースするたびに、実際にその古地図の場所を歩きながら機能を実演するショート動画を撮影し、SNS・Web 公開ページに掲載する
- AI の物語生成コスト（プロンプト長・モデル選定・キャッシュ戦略）を見直し、コストを下げられた分をユーザー還元（無料枠の拡大、サブスクリプション価格の引き下げ）に回し、「業界標準より太っ腹」という評判を作る

共通：無料／有料の再設計（3 哲学の統合）

- 現状すべて無料で提供している古地図カタログを、「フェーズ1 で選んだ主力エリア+2~3 エリア」を恒久無料の“完成された小さな体験”（ジョブス型）とし、それ以外のエリアを地域パックとして課金対象にする境界線を明確に引き直す
- 有料の地域パックには、単なる「エリア追加」ではなく、観光協会等とのタイアップによる「他では手に入らない公式コンテンツ」（ティール型の独占資産）を必ず1 つ以上含める
- リファラルで獲得した新規ユーザーに対しては、招待した側・された側の双方に「無料エリアの体験がすでに完璧である」ことを最初に見せ切り、その上で有料パックへの導線を提示する順序を徹底する
- Firebase Analytics で、エリアごとの利用率・課金率・リファラル経由の定着率を計測するダッシュボードを整備し、「どのエリアが独占できているか」「どの哲学の施策が効いているか」を定量的に見直すサイクルを月次で回す

20-6. 収益化する方法一覧：3 人の哲学から導く具体的なマネタイズ手法

20-1～20-5 節では、3 人の哲学を「無料／有料の分け方」「プロダクトの魅力の作り方」「ユーザーの見立て方」という 3 つの観点、および実践チェックリストとして整理しました。本節では視点をさらに絞り、「実際にどうやって Komap でお金を生むか」という収益化手法そのものを、哲学ごとに具体的なリストとして列挙します。

■ スティーブ・ジョブズ型：プレミアム・完成度重視のマネタイズ

- **地域パックの買い切り課金**：サブスクではなく 1 エリアごとの一括購入（例：500～1,200 円）とし、値引きセールは一切行わない。「安くするために質を下げる」余地そのものを作らない。
- **「デザイナーズエディション」古地図**：著名なイラストレーター・郷土史家監修の限定古地図を高単価（例：2,000 円前後）で販売し、通常パックとは別格の「完成品」として位置づける。
- **広告枠を最初から作らない**：無料化・広告収益化には手を出さず、体験の一貫性とブランドの高級感を優先する。
- **年間契約のみのプレミアムプラン**：月額課金は提供せず、「じっくり使い込む人だけ」を対象にした年払いのみのサブスクリプションにする。
- **実物とのセット販売**：御朱印帳アプリの記録と連動した「本物の御朱印帳」を EC で同時販売し、Apple のハード×ソフト垂直統合に相当する体験を作る。

■ ピーター・ティール型：独占・ニッチ支配のマネタイズ

- **自治体・観光協会との独占ライセンス契約**：特定エリアの「公式古地図アプリ」として年間契約を結び、他アプリが参入しにくい既成事実を作る。
- **教育機関向けライセンス（B2B2C）**：学校の郷土学習教材として、学級・学校単位の年間サブスクリプションで販売する。
- **デジタルスタンプラリーの運営受託**：地域イベント単位で、企業・自治体からラリー機能の構築・運営を業務委託として受注する。
- **匿名人流データの有償レポート化**：蓄積した「人気の史跡・滞在時間帯」等の匿名集計データを、自治体・地域事業者向けレポートとして販売する。
- **独自古地図アーカイブの OEM ライセンス**：他社が持たない独自デジタル化史料を、他の観光アプリ・出版社へライセンス提供する。

■ イーロン・マスク型：ミッション駆動・規模拡大のマネタイズ

- **リファラル報酬型フリーミアム**：友人を招待して一緒に歩くと、双方に地域パックまたは AI

生成回数を無料で解放する。

- **歩数連動のアプリ内通貨**：「歩くほど貯まるポイント」を発行し、将来的にグッズ・限定コンテンツと交換できる仕組みにする。
- **クラウドファンディング型の新エリア追加**：新しい古地図エリアの制作費を支援者から募り、達成すれば支援者に先行アクセス権を付与する。
- **低価格・広範囲の月額サブスク**：AI 生成コストの第一原理的な見直しで下げた原価を、業界最安値クラスの月額価格に還元し普及速度を最大化する。
- **ミッション共感型の法人スポンサーシップ**：地域創生ミッションに共感する企業を「協賛古地図」として 1 エリア 1 社限定で掲載し、ユーザー体験を損なわない範囲で協賛収入を得る。

いずれの手法も単独では機能しません。20-4 節の「フェーズでの使い分け」と同様に、まずティール型で独占するニッチを決め、ジョブス型で体験を磨き込み、マスク型で拡大するという順序で組み合わせることで、Komap という 1 つのプロダクトの中に矛盾なく実装できます。

第 21 章 費用構造とマーケティング ROI の詳細分析

これまでの章では「作り方」と「収益化・マーケティングの考え方」を扱ってきました。本章では、実際に個人開発者が直面する**具体的な費用（コスト）**と、マーケティングに投じた労力・お金に対する**リターン（ROI）**を、数値シミュレーションを交えて詳細に分析します。

個人開発の地図ゲームアプリは、自前サーバーを持たない構成（Google Maps Platform・Firebase・AI API）を取ることが多いため、コストの大半は「従量課金」です。これは「最初はほぼ無料で始められるが、ユーザーが増えるほど費用も増える」という構造を意味します。マーケティングを成功させてユーザーが急増した瞬間に、コスト構造を理解していないと利益どころか赤字に転落するリスクがあるため、本章を必ず開発の初期段階で読み込んでおくことをお勧めします。

注意：本章に記載する単価・料金は、執筆時点で公開されている情報をもとにした**概算・試算のための例示**です。Google Maps Platform、Firebase、各種 AI API の料金体系は改定される頻度が高いため、実装・予算計画の前には必ず各社の最新の公式料金ページを確認し、本章の数値はあくまで「コストの内訳と考え方の型」を理解するための参考値として扱ってください。

21-1. コスト構造の全体像：固定費 vs 変動費

地図ゲームアプリのコストは、大きく「固定費」と「変動費（ユーザー数・利用量に比例するコスト）」に分けられます。

固定費（ユーザー数に関わらず発生）

- Apple Developer Program 年会費（年間 \$99）
- ドメイン取得・維持費（年間 数千円程度）

- （任意）デザイン素材・古地図史料の購入費
- （任意）法人化している場合の税務・会計コスト

変動費（ユーザー数・利用量に比例）

- Google Maps Platform（地図の読み込み回数に応じた課金）
- Firebase（Firestore の読み書き回数、Storage の容量・転送量、Functions 実行回数）
- AI API（生成した物語の文字数＝トークン数に応じた課金）
- （広告を出す場合）広告費（CPC・CPI 課金）

個人開発において最も重要なのは、「**1 人のアクティブユーザーが 1 ヶ月にどれだけの変動費を発生させるか（ユーザーあたり原価）**」を早い段階で概算しておくことです。これが分からないまま無料ユーザーを増やすマーケティングだけを行うと、「ユーザーは増えたが、増えるほど赤字が拡大する」という状態に陥りかねません。

21-2. Google Maps Platform 費用の内訳

主な課金対象

- **Maps SDK for iOS（動的地図の読み込み）**：アプリ内で地図画面を開くたびに「1 回の地図読み込み」としてカウントされる
- **Geocoding API**（住所⇄緯度経度の変換。史跡の登録作業などで使用する場合）
- **Static Maps API**（プレビュー画像として静止画の地図を生成する場合。第 4 章で紹介した御朱印一覧のプレビュー地図のような用途）

コストシミュレーションの考え方

Google Maps Platform には、月間一定額までの無料利用枠が用意されています（具体的な上限額・対象 SKU は変更されることがあるため要確認）。無料枠を超えた分は、1,000 回の地図読み込みあたり数ドル程度の従量課金が発生する、という料金体系が基本です。

試算のために、次のような前提を置きます（あくまで例示です）。

- 1 人のアクティブユーザーが 1 回のセッションで地図画面を平均 3 回開く（起動時・古地図切り替え時・記録画面遷移時など）
- 1 人のアクティブユーザーが 1 日あたり平均 1.2 セッション利用する
- 地図読み込み単価を仮に「1,000 回あたり \$7」とする

この前提で、月間アクティブユーザー（MAU）数ごとの地図読み込み回数と概算費用は次のようになります。

MAU	月間セッション数	月間地図読み込み回数	概算費用（無料枠考慮前）
100	3,600	10,800	約 \$76
1,000	36,000	108,000	約 \$756
10,000	360,000	1,080,000	約 \$7,560
100,000	3,600,000	10,800,000	約 \$75,600

この試算から分かる重要な示唆は、「**地図の読み込み回数**」が**ユーザー数**に対して**ほぼ線形にコストを増加させる**ということです。したがって、コスト最適化の主戦場は「1 セッションあたりの地図読み込み回数をどれだけ減らせるか」になります。

コスト削減の実践手法

- **地図インスタンスの使い回し**：画面遷移のたびに新しい GMSMapView を生成せず、可能な限り同一インスタンスを使い回し、カメラ位置だけを更新する
- **プレビュー用途は Static Maps API または自前のキャッシュ画像に切り替える**：御朱印一覧など、動的な操作が不要なプレビューは、動的地図（動的読み込みが都度課金される）ではなく、一度生成してキャッシュした静止画に置き換えることで、読み込み回数そのものを発生させない
- **オフライン・非表示時は地図の更新を停止する**：バックグラウンド遷移時やアプリが非アクティブな間は、地図の描画・位置更新を一時停止し、無駄な読み込みを避ける
- **予算アラートを必ず設定する**：Google Cloud Platform の請求先アカウントに、月額上限に対する通知（例えば予測費用が\$50、\$100 を超えたら通知）を必ず設定し、想定外の急増を早期に検知する

21-3. Firebase 費用の内訳

Firestore（データベース）

Firestore は「読み取り回数」「書き込み回数」「削除回数」「保存データ量」「ネットワーク転送量」に応じて課金されます。無料枠（Spark プラン）内でも、1 日あたりの読み書き回数に上限があり、それを超えると Blaze プラン（従量課金）への移行が必要になります。

試算のための前提（例示）：

- 読み取り単価：概算で 10 万回あたり \$0.06
- 書き込み単価：概算で 10 万回あたり \$0.18
- 1 人のユーザーが 1 日に Firestore へ平均 30 回の読み取り・5 回の書き込みを発生させる（一覧表示、いいね・コメントの購読、記録の保存など）

MAU	月間読み取り回数	月間書き込み回数	概算費用
100	90,000	15,000	無料枠内に収まる可能性が高
1,000	900,000	150,000	約 \$0.8(読み取り)+約 \$0.27(書き込み)約 \$1.1
10,000	9,000,000	1,500,000	約 \$8.4
100,000	90,000,000	15,000,000	約 \$84

Firestore は、Google Maps Platform と比べると 1 ユーザーあたりの単価が非常に小さいことが分かります。ただし、onSnapshot によるリアルタイムリスナー（いいね数のリアルタイム表示など）を無制限に張り続けると、意図せず読み取り回数が急増することがあるため注意が必要です。

コスト削減の実践手法

- 一覧画面での購読は、画面が表示されている間だけに限定し、非表示になったら onSnapshot のリスナーを解除する
- いいね数・コメント数のような「集計値」は、書き込みのたびにコレクション全体を数え上げるのではなく、親ドキュメントにカウンタフィールドを持たせ、Cloud Functions のトリガーでインクリメント／デクリメントする設計にすると読み取りコストを大幅に下げられる
- ページネーション（limit + startAfter）を徹底し、一覧の全件を一度に読み込まない

Firestore Storage (画像保存)

Storage は「保存容量」と「ダウンロード（転送）量」に応じて課金されます。

- 保存容量単価：概算で GB あたり月\$0.026
- ダウンロード単価：概算で GB あたり\$0.12

御朱印・投稿写真の画像（圧縮後、平均 300KB 程度と仮定）を 1 人のユーザーが月 10 枚保存し、それを他のユーザーが閲覧する（1 枚あたり平均 5 回表示されると仮定）場合の試算：

MAU	月間新規保存量	保存累計費用 (12 ヶ月)	月間ダウンロード量	月間ダウンロード費用
1,000	約 3GB	約 \$0.9	約 15GB	約 \$1.8
10,000	約 30GB	約 \$9	約 150GB	約 \$18
100,000	約 300GB	約 \$90	約 1,500GB	約 \$180

Storage のコストで最も効くレバーはアップロード前の圧縮・リサイズです。第 8 章・第 4 章で紹介した 1024px 程度への縮小を徹底するだけで、保存容量・転送量の双方を数分の 1 に削減できます。

Cloud Functions (サーバーサイド処理)

TestFlight 招待メールの自動送信、いいね数のカウンタ更新など、サーバーサイド処理は Cloud

Functions で実装します。課金は「呼び出し回数」「実行時間（CPU・メモリ）」「ネットワーク転送量」の組み合わせです。無料枠（月 200 万回呼び出しなど）の範囲内で収まるケースが多いですが、Blaze プランへの切り替え自体が前提条件になる点は第 8 章で触れた通りです。

Firestore Hosting (Web 版の配信)

Web 版アプリの静的ファイル配信は、無料枠（月 10GB の転送量など）で個人開発の初期段階は十分にまかなえることが多く、優先度の高いコスト最適化対象にはなりにくい領域です。

21-4. AI 生成コストの内訳

トークン単価の考え方

OpenAI などの LLM API は、「入力トークン数」と「出力トークン数」にそれぞれ異なる単価が設定されています。低コストなモデル（例：gpt-4o-mini 相当のモデル）を使う場合の概算単価は次の通りです（例示）。

- 入力：100 万トークンあたり \$0.15
- 出力：100 万トークンあたり \$0.60

第 7 章のプロンプト設計（入力 200 トークン程度、出力 500 文字＝日本語ではおよそ 300～400 トークン程度と仮定）で、1 回の物語生成にかかるコストを試算します。

入力コスト = $200 \text{ トークン} \div 1,000,000 \times \$0.15 \div \$0.00003$

出力コスト = $400 \text{ トークン} \div 1,000,000 \times \$0.60 \div \$0.00024$

1 回あたりの生成コスト $\div \$0.00027$ （日本円で 0.04 円程度）

一見すると無視できるほど小さい金額ですが、ユーザー数とチェックポイント数が増えると無視できない規模になります。1 人のユーザーが 1 ヶ月に平均 20 回の物語生成を行うと仮定した場合の試算は次の通りです。

MAU 月間生成回数概算費用（キャッシュなし）

100 2,000 約 \$0.54

1,000 20,000 約 \$5.4

10,000 200,000 約 \$54

100,000 2,000,000 約 \$540

コスト削減の実践手法

- **生成結果のキャッシュ**：同じ史跡・同じ切り口の物語は、ユーザーごとに毎回再生成せず、一度生成した結果を Firestore 側にキャッシュして再利用する（第 7 章参照）。同一史跡への複数ユーザーからのアクセスに対してキャッシュを共有できれば、生成回数そのものを大幅に削減できる

- **バリエーションの事前生成**：人気の高い史跡については、深夜バッチ処理などであらかじめ数パターンの物語を生成しておき、ランダムに提示する。リアルタイム生成の回数を減らしつつ「毎回違う物語」という体験は維持できる
- **プロンプトの圧縮**：不要な前置き・繰り返しの指示文をプロンプトから削り、入力トークン数を最小化する
- **出力の文字数上限を明示的に指定する**：max_tokens のようなパラメータで出力上限を設定し、想定より長い（＝高コストな）出力が生成されるのを防ぐ

21-5. ユーザー数別・月額総コストシミュレーション

21-2～21-4 の試算を合算すると、地図ゲームアプリの月額総コスト（変動費部分）のおおよその規模感は次のようになります。

MAU	Google Maps	Firestore	Storage	AI 生成	月額合計（概算）	1 ユーザーあたり原価
100	約\$76	ほぼ無料	ほぼ無料	約\$0.5	約 \$77	約 \$0.77
1,000	約\$756	約\$1.1	約\$2.7	約\$5.4	約 \$765	約 \$0.77
10,000	約\$7,560	約\$8.4	約\$27	約\$54	約 \$7,649	約 \$0.76
100,000	約\$75,600	約\$84	約\$270	約\$540	約 \$76,494	約 \$0.76

この試算から見える最も重要な結論は、**総コストの 9 割以上を Google Maps Platform の地図読み込み費用が占める**という点です。Firestore・Storage・AI 生成のコストは、地図ゲームというジャンルにおいては相対的に小さく、最適化の優先順位としては「地図の読み込み回数をどう減らすか」が最重要課題になります（21-2 で紹介した削減手法を必ず実践してください）。

また、1 ユーザーあたりの原価がおおよそ\$0.7～0.8 程度で安定する（規模の経済がそれほど働かない）ことも分かります。これは、後述する収益化モデルの価格設定・損益分岐点分析の土台になる、個人開発者にとって非常に重要な数字です。

21-6. マーケティング費用対効果（ROI）の分析

CAC（顧客獲得コスト）と LTV（顧客生涯価値）

マーケティング施策の ROI を評価する基本の型は、**CAC（Customer Acquisition Cost：1 人のユーザーを獲得するためにかった費用）**と **LTV（Life Time Value：1 人のユーザーが生涯にわたってもたらす収益）** 比較です。一般的に LTV が CAC の 3 倍以上であれば健全な収益構造とされます。

チャンネル別の CAC 試算

チャンネル	想定 CAC (1 インストール)	特徴
SNS オーガニック投稿	ほぼ\$0（労働時間のみのみ）	低コストだが再現性・スケール性は運次

チャンネル	想定 CAC (1 インストール)	特徴
(Before/After 動画等)		第
個人開発者コミュニティ・Product Hunt 掲載	ほぼ\$0 (労働時間のみのみ)	アーリーアダプター層に強いが規模は限定的
地域メディア・プレスリリース	\$0～数万円 (配信サービス利用時)	地域性の強いアプリと相性が良く、CAC が非常に低くなりやすい
App Store 広告 (Apple Search Ads)	\$1～\$3 程度 (競合状況によって変動)	ASO と組み合わせることで効率が上がる。予算のコントロールがしやすい
SNS 広告 (Meta 広告・X 広告など)	\$1～\$5 程度 (ターゲティング精度による)	短期間でボリュームを作れるが、地図ゲームのようなニッチ層への精密なターゲティングが鍵
インフルエンサー起用	案件による (無償の相互紹介～数万円規模まで幅広い)	郷土史・観光系のマイクロインフルエンサーとの親和性が高い

個人開発者は、まず **CAC がほぼゼロに近いチャンネル (SNS オーガニック、コミュニティ、地域メディア、リファラル)** を最大限使い切ってから、余力があれば有料広告 (Apple Search Ads 等) を小予算でテストする、という順番を取ることを強くお勧めします。

LTV の試算

第 13 章で提案した「フリーミアム + 地域パック課金」モデルを前提に、LTV を試算します。

- 無料ユーザーのうち、地域パック (例：1 パック 600 円) を購入する割合 (課金率) を仮に 3% と設定
- 課金ユーザーのうち、平均 1.5 パックを購入すると仮定
- 平均継続利用期間を 6 ヶ月と仮定

$$1 \text{ ユーザーあたり平均売上} = 3\% \times 600 \text{ 円} \times 1.5 \text{ パック} \div 6 \text{ ヶ月} = 27 \text{ 円}$$

この数字だけを見ると小さく感じますが、21-5 で試算した 1 ユーザーあたり原価 (約\$0.7～0.8、円換算でおおよそ 100～120 円) と比較すると、**現状のフリーミアムモデルの単価設定では、マス (多くのユーザー) を集めるほど赤字が拡大する**という重要な事実が見えてきます。

この試算が示す実践的な示唆は次の 3 点に集約されます。

1. **無料ユーザーの原価 (特に Google Maps 読み込み費用) を抑える技術的な工夫 (21-2 参照) が、収益化モデルの成否を左右するほど重要である**
2. **課金率 3%・単価 600 円という前提のままでは、広く浅いマーケティングよりも、課金率を引き上げる施策 (地域パックの魅力を上げる、サブスクリプションへの誘導を強化する) の**

優先度が高い

3. ティール型戦略（第 20 章）で提案した「独占できるニッチ×観光協会タイアップの公式コンテンツ」のように、通常のパックより高い単価を正当化できる商品ラインを持つことが、LTV 改善の近道になる

損益分岐点 (Break-even) のシミュレーション

上記の前提（原価:1 ユーザーあたり約 110 円/月、売上:1 ユーザーあたり平均 27 円/月〈フリーミアムのみ〉）のままでは、ユーザー数が増えるほど赤字が拡大する構造です。損益分岐点に到達するためのレバーは主に 3 つです。

- **原価を下げる**（21-2～21-4 のコスト削減手法をすべて実践する。特に地図読み込み回数の削減が最重要）
- **課金率を上げる**（3%→10%など。第 20 章のジョブス型「体験の完成度」戦略、ティール型「独占コンテンツ」戦略が有効なレバー）
- **単価を上げる**（600 円のパックに加え、観光協会タイアップの 1,200 円プレミアムパックを用意するなど）

例えば、原価を 21-2 の削減手法で 30%圧縮（約 77 円/月）し、課金率を 8%、平均単価を 800 円に改善できた場合の試算は次の通りです。

改善後の 1 ユーザーあたり平均売上 = 8% × 800 円 × 1.5 パック ÷ 96 円

原価 77 円に対して売上 96 円となり、ここで初めて黒字構造に転換します。「**まずマーケティングでユーザーを増やす**」のではなく、「**原価と課金率・単価の構造を黒字化してから、マーケティングで規模を拡大する**」という順序を守ることが、個人開発における最大のリスク管理です。

21-7. マーケティング施策ごとの ROI まとめ

以上の分析を踏まえ、代表的なマーケティング施策を「投下コスト」「期待リターン」「地図ゲームアプリとの相性」の観点でまとめます。

施策	投下コストの目安	期待リターン	相性・注意点
SNS での Before/After 動画投稿	ほぼ\$0（撮影・編集の労力）	バズれば数千～数万インプレッション、低いが継続的なインストール	透過度アニメーションのデモ機能を使えば撮影コストも低い。再現性は運次第なので継続投稿が前提
リファラルプログラム	実装コスト（開発工数招待経由ユーザーは定着 + 原資（無料解放分の機会損失）	率が低い傾向。CAC が質的にコンテンツ原価のみに圧縮される	マスク型戦略の中核。実装優先度を上げる価値がある

Apple Search 数万円～（予算上限を即効性のあるインストー ASO（キーワード・スクリーンショ

施策	投下コストの目安	期待リターン	相性・注意点
Ads	自分で設定可能)	ル数増加	ット) が固まってから投下しないと、CAC が割高になりやすい
サブスクリプションのインポート価格施策	ほぼ\$0 (設定変更のみ)	初回コンバージョン率の改善	期間限定であることを明示し、恒常的な値引きにしない (ジョブス型のプレミアム・ポジショニングを崩さない)

21-8. Komap 適用：費用・ROI シミュレーションの実践チェックリスト

- ☐ Google Cloud Platform・Firebase の請求先アカウントに、月額\$50・\$100・\$300 の3段階で予算アラートを設定する
- ☐ GoogleMapRepresentable で地図インスタンスを画面遷移ごとに再生成していないか確認し、再生成している箇所があればインスタンスの使い回しに書き換える
- ☐ StampListView のプレビュー地図 (第4章参照) が動的な GMSMapView のままであれば、静止画キャッシュ方式への置き換えを検討し、地図読み込み回数を削減する
- ☐ Firestore の onSnapshot 購読箇所 (いいね・コメントのリアルタイム表示) を洗い出し、画面が非表示になったタイミングで確実にリスナーを解除できているか確認する
- ☐ sharedTrips のいいね数を「毎回コレクションを数え上げる」実装のままにしていないか確認し、カウンタフィールド+Cloud Functions トリガー方式への移行を検討する
- ☐ AIHistoryService が生成した物語をキャッシュせず毎回再生成していないか確認し、史跡単位でのキャッシュ・複数バリエーションの事前生成に切り替える
- ☐ 現状の想定 MAU・課金率・単価をもとに、本章 21-6 の計算式に自社の実測値を当てはめ、「今、黒字構造になっているか」を月次で確認するスプレッドシートを作成する
- ☐ 損益分岐点に達していない場合、マーケティング予算を投下する前に「原価削減」「課金率改善 (体験の磨き込み)」「単価改善 (タイアップ限定コンテンツ)」のどれを優先するかを、本章の試算に基づいて意思決定する
- ☐ CAC がほぼゼロのチャンネル (SNS オーガニック、地域メディア、リファラル) を先に使い切ってから、Apple Search Ads など有料チャンネルの小規模テストに進む順序を徹底する

第 22 章 外部機器連携・写真管理・表示品質の磨き込み

第 17 章のケーススタディでは、最初のリリースにたどり着くまでの拡張の歴史を振り返りました。本章では、それに続く「2 つめ以降の週末」で実際に追加した機能を、実装の勘所とともに紹介します。派手な新機能よりも、こうした地道な拡張・磨き込みの積み重ねこそが、リリース後のアプリを成長させる原動力になります。

22-1. 連携カメラ・連携プリンター：外部の IoT 機器と連携する

M5Stack のような IoT 機器を、自宅 Wi-Fi 上の「もう 1 つのカメラ」「もう 1 つのプリンター」としてアプリに組み込む機能を追加しました。設計のポイントは次の 3 つです。

1. 設定はホスト名だけ、パスはアプリ側で組み立てる

「設定」画面ではユーザーにホスト名/IP（例：m5web.local）だけを入力してもらい、実際の API パス（/api/print/photo など）はアプリ側の定数として管理します。ユーザーが古い設定値（別の API 仕様を試していた時の値）を消し忘れていても、URLComponents で一度分解してスキーム・ホスト・ポートだけを取り出し直すことで、意図しない URL が組み上がるのを防いでいます。

```
private static func hostBase(from input: String) -> String? {
    let trimmed = input.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !trimmed.isEmpty else { return nil }
    let withScheme = trimmed.lowercased().hasPrefix("http")
        ? trimmed : "http://¥(trimmed)"
    guard let components = URLComponents(string: withScheme), let host = components.host else { return nil }
    var base = "¥(components.scheme ?? "http")://¥(host)"
    if let port = components.port { base += "¥(port)" }
    return base
}
```

2. 転送方式を複数用意し、相手の API 仕様に合わせて選べるようにする

連携プリンターの API には、「画像データを直接受け取るタイプ」（multipart/form-data での POST）と「画像の URL だけを受け取り、自分で取得しにいくタイプ」（GET にクエリパラメータで URL を渡す）の 2 通りが実務では存在します。後者のためには、画像を一時的にクラウドストレージへアップロードして URL を発行する必要があります。両対応にしておくことで、相手の実装に合わせて設定を切り替えるだけで済むようにしました。

```
switch AppSettings.printerTransferMode {
case .direct:
    try await sendViaMultipartPOST(data: data, format: format, base: base)
case .hostedURL:
    try await sendViaHostedURL(data: data, format: format, base: base, existingURL: cloudURL)
}
```

3. 「送信しました」と表示されるのに実際には届かない問題への対応

IoT 機器との通信でありがちな落とし穴として、`URLError.networkConnectionLost`（「network connection was lost」）が挙げられます。多くの場合、次の 2 つの組み合わせで解決します。

- `Connection: close` ヘッダーを明示し、iOS 側にコネクションの使い回し（keep-alive）をさせない
- ボディ付き POST で自動付与される `Expect: 100-continue` ヘッダーを、`Expect` ヘッダーを自分で（空文字で）明示することで抑止する（簡易的な ESP32/Arduino 系の Web サーバーはこのヘッダーに対応できず、接続を切ってしまうことが多い）

さらに、HTTP ステータスコードを確認せずに「リクエストが例外を投げなければ成功」と判定してしまうと、実際にはプリンター側が 404 や 500 を返していても「送信しました」と表示されてしまいます。レスポンスの `HTTPURLResponse.statusCode` を必ず確認し、2xx 以外は失敗として扱うようにしましょう。

```
let (_, response) = try await URLSession.shared.data(for: request)
if let http = response as? HTTPURLResponse, !(200..<300).contains(http.statusCode) {
    throw PrintError.requestFailed("HTTP ステータス ¥(http.statusCode)")
}
```

エラーメッセージには、可能な限り「原因」を含めることをお勧めします。「送信に失敗しました」だけでは、ユーザー（そして未来の自分）はどこから調査すればいいかわかりません。通信エラーの `localizedDescription` や HTTP ステータスコードをそのまま画面に出すだけで、トラブルシューティングの速度が大きく変わります。

22-2. 投稿写真の管理：フリック・削除・非公開

写真は「撮って終わり」ではなく、後から見返し、時には削除したり公開範囲を調整したりするものです。既存の 1 枚だけのプレビュー画面を、次のように拡張しました。

同じグループ内の写真をフリックで移動できるようにする

SwiftData の `@Query` を使い、同じ `walkRouteID` を持つ写真を動的にフィルタして取得し、`TabView` の `.page` スタイルでページングします。ポイントは、**表示対象の一覧を親から渡された固定配列ではなく、`@Query` で直接監視すること**です。これにより、シートの中で 1 枚削除しても、一覧が自動的に更新されて隣の写真にシームレスに移動できます。

```
init(post: WalkPhotoPost) {
    _selectedPostID = State(initialValue: post.id)
    if let walkRouteID = post.walkRouteID {
        _posts = Query(
            filter: #Predicate<WalkPhotoPost> { $0.walkRouteID == walkRouteID },
            sort: ¥WalkPhotoPost.postedAt
        )
    } else {
        let postID = post.id
        _posts = Query(filter: #Predicate<WalkPhotoPost> { $0.id == postID })
    }
}
```

「非公開にする」は削除ではなく、公開データからの除外というフラグにする

「みんなの時空旅」に公開中の記録であっても、特定の 1 枚の写真だけは公開したくない、という要望は自然に発生します。これを実現するために、写真 1 枚ごとに `isHiddenFromSharing` という真偽値を持たせ、公開データを作り直す処理 (`setPubliclyShared`) の中でこのフラグが立った写真を除外するだけにしました。

```
for post in photoPosts where post.photo != nil && !post.isHiddenFromSharing {  
    // 公開データ (sharedTrips 配下) に含める  
}
```

削除・非公開化のどちらも、それが属する時空旅が既に公開済みであれば、公開データを都度作り直す (再同期する) 処理を呼び出す必要があります。この「編集のたびに公開データを再同期する」というパターンは、本書を通じて何度も登場しています。編集操作を追加するたびに「これは公開データにも影響するか？」を自問する習慣をつけておくと、公開・非公開の整合性が崩れる不具合を防げます。

22-3. 地図表示の継続的な磨き込み

古地図オーバーレイの表示品質はリリース後もユーザーの反応を見ながら細かく調整します。

「霧の演出」が強すぎて古地図が見えない、というフィードバック

第 4 章で紹介した「歩いた場所だけくっきり見せる」演出 (宝探し効果) は、実装当初、ぼかしの強さ (ガウスぼかし半径 14) と、まだ通っていない場所の不透明度の下限 (0.6) の組み合わせが強すぎて、「古地図がぼんやりとしか見えない」というフィードバックにつながりました。対策として、ぼかし半径を 5 まで弱め、不透明度の下限を 0.88 まで引き上げました。ゲーム性 (宝探し感) と実用性 (古地図として見やすいこと) のバランスは、こうした数値のチューニングによって初めて着地点が見つかります。最初の実装値を「仮の値」として割り切り、リリース後に実際のフィードバックで追い込んでいく姿勢が重要です。

ズーム・パン後にオーバーレイが消える不具合

Google Maps SDK には、大きくズーム・パンした直後に `GMSGroundOverlay` のテクスチャが GPU 側で失われたまま再描画されない、という既知の癖があります。対策は、カメラの移動が収まった (idle) タイミングで、オーバーレイを重い再デコードなしに一旦外して貼り直すことです。

```
func mapView(_ mapView: GMSMapView, idleAt position: GMSCameraPosition) {  
    currentOverlay?.map = nil  
    currentOverlay?.map = mapView  
}
```

このような「SDK の内部実装に起因する消失・再描画不良」は、実機で長時間・様々な操作を試してみないと見つからないことが多く、机上のテストだけでは気づけません。リリース後、実際のユーザーの操作パターン (大きくズームする、地図を素早くドラッグするなど) から不具合が報告されたら、「操作の直後に強制的に再描画し直す」という対症療法的な対策から試してみる価値があります。

22-4. Web 版をマーケティングの入り口にする

第 II 部で扱ったマーケティングの考え方を、実際に Web 公開ページ (komap.ktrips.net) に落とし込んだ実例を紹介します。

Web 版は「見るだけ」の体験、iOS アプリが「本体験」——役割を明確に分ける

未サインインの訪問者向けの Web 公開ページは「みんなの時空旅」を眺められる入り口として設計されていますが、それだけで終わらせず **iOS アプリ移行を強く後押しする導線**を組み込みます。

- ヘッダーの「使い方」ボタンから開くモーダルでは、Web 版でできることはあえて 1~2 文に圧縮し、iOS アプリでできること（実際に歩いて古地図を発見する、AI が物語を語る、御朱印を集める、外部機器と連携する）をカード形式で強調しています。「この Web ページで見られるのは、iOS アプリで生まれた記録の“一部”です」という一文で、Web 版がゴールではなく入り口であることを明示し、末尾に「Google でサインインして、iOS アプリを始める」ボタン（サインインすると、既存の仕組みで TestFlight の招待メールが自動送信される）を置いています
- 当初は公開ページ下部にも独立した「iOS アプリをダウンロード」ボタンを置いていましたが、同じ画面に似た誘導ボタンが並ぶと逆にどれを押せばいいかわりにくくなったため、後日取り除きました。誘導は「使い方」モーダル 1 箇所に集約し、そこでしっかり訴求する方針に落ち着いています。マーケティング施策は追加するほど良いとは限らず、効果が重なる導線は思い切って削る判断も必要です

書いている本そのものを、マーケティングコンテンツにする

本書は Kindle での出版を目指して執筆されていますが、完成を待たずに、執筆中の原稿の冒頭部分を「Komap の作り方 Kindle（一部無料）」というボタンから Web 公開ページ上でそのまま読めるようにしました。原稿全文をクライアントに配信するのではなく、あらかじめ冒頭部分だけを切り出した専用ファイルを配信することで、「続きが気になる」という興味を安全に喚起しつつ、原稿全体の流出を防いでいます。

```
fetch("/kindle-preview.md")
  .then((response) => response.text())
  .then((markdown) => setHtml(marked.parse(markdown) as string));
```

この施策は、第 14~16 章で紹介した「開発の過程そのものをコンテンツにする」というマーケティング手法の延長線上にあります。個人開発者にとって、開発記録・執筆中の原稿・裏側の試行錯誤は、追加のコストをほとんどかけずに用意できる最も飾らない形のマーケティングコンテンツです。

22-5. パフォーマンスの継続的な改善——「動いているから十分」で終わらせない

機能追加が一段落した後も、実際の利用シーンを想像しながらパフォーマンスを見直す作業は続きます。ここでは、リリース後に見つかった 2 つの具体的な事例を紹介します。

Web 版：無関係な再描画が、地図をまるごと作り直させていた

React では、props に渡す配列やオブジェクトを useMemo で固定しておかないと、コンポーネントが再描画されるたびに「新しい配列」が作られ、それを受け取った子コンポーネントの useEffect が「依存関係が変わった」と誤検知することがあります。時空旅の詳細画面では、地図に重ねる史跡チェックポイントの一覧を次のように毎回その場で計算していました。

```
<TripMapView
  latitudes={trip.latitudes}
  longitudes={trip.longitudes}
  oldMap={oldMap}
  checkpoints={sitesForOverlay(oldMap?.id ?? null)} // 呼ぶたびに新しい配列
/>
```

いいね・コメントのリアルタイム更新やコメント欄への入力など、**地図と無関係な理由で画面が再描画されるたびに**、この新しい配列が地図コンポーネント側の useEffect を発火させ、古地図オーバーレイの再取得（画像をネットワークから読み直す）・マーカーとポリラインの作り直し・ズームの再調整を毎回引き起こしていました。「地図の表示に時間がかかる」というユーザーからの指摘は、まさにこれが原因でした。

```
const checkpoints = useMemo(() => sitesForOverlay(oldMap?.id ?? null), [oldMap]);
```

useMemo で固定するだけの、1 行の修正です。修正の効果は、ブラウザの開発者ツールで GroundOverlay のコンストラクタ呼び出し回数を実際に数えて検証しました——「直しました」で終わらず、狙った再描画が本当に減ったかを数値で確認する一手間が、こうした性能問題では特に重要です。

iOS 版：フル解像度画像の縮小処理が、メインスレッドをブロックしていた

古地図を切り替えるたびに、3000px を超えることもあるフル解像度の画像を 1600px まで縮小する処理を、これまでメインスレッドで同期的に実行していました。この処理自体は「見た目には」問題なく動きますが、実行中は UI の描画が一瞬止まるため、「地図タブを開いた瞬間」や「歩行記録開始時に古地図が自動表示される瞬間」に、体感できる「もたつき」として現れていました。

対策は次の 2 段階です。

- 縮小結果を古地図 ID ごとにキャッシュし、2 回目以降の表示では即座に使い回す
- 未キャッシュの初回は、まず画像なし（枠だけ）のオーバーレイを即座に表示しておき、縮小処理はバックグラウンドスレッドで行ってから、完了時に画像だけを差し替える

```
if let cached = Self.singleOverlayImageCache[overlayID] {
  // キャッシュ済みなら即座に使う
} else {
  // まず枠だけ表示 → バックグラウンドで縮小 → 完了後に icon を差し替え
  DispatchQueue.global(qos: .userInitiated).async { ... }
}
```

「メインスレッドで重い処理をしない」という原則自体は目新しいものではありませんが、リリース初期にはこの規模の画像を想定していなかったため当初のコードでは見落とされていました。ユーザ

一からの「時間がかかる」という一言をきっかけにコードを読み返して初めて気づけた典型例です。

iOS 版：GPS 更新のたびに処理が、歩行時間に比例して重くなっていた

歩行記録中は数秒おきに GPS の新しい座標が届き、そのたびに「軌跡を滑らかに描き直す」「古地図の“めくれ”演出を合成し直す」処理が走ります。当初の実装は、どちらも新しい座標が 1 件増えるたびに蓄積した軌跡全体を対象に計算をやり直していました。

// 修正前：新しい点が 1 つ増えるだけで、蓄積済み全体を毎回スムージングし直す

```
let path = GMSMutablePath()
```

```
Self.smoothedTrailCoordinates(live).forEach { path.add($0) } // live は記録開始からの全座標
```

歩き始めた直後はほとんど気づきませんが、30 分・1 時間と歩き続けるうちに live（記録済みの全座標）の要素数がどんどん増え、GPS 更新 1 回あたりの計算量も比例して増え続けます。「歩き始めは快適だったのに、長く歩くとだんだんカクつく」という、時間が経たないと表面化しない典型的な性能劣化です。

対策は、新しく増えた区間だけを対象に、既存の結果へ差分で追記することです。

// 修正後：前回の終端点を起点に、新しく増えた区間だけスムージングして追記

```
path.removeLastCoordinate()
```

```
for index in (previousCount - 1)..  
(live.count - 1) {  
    let p0 = live[index]  
    let p1 = live[index + 1]  
    path.add( /* p0 と p1 のスムージング済み中間点 */ )  
}  
path.add(live[live.count - 1])
```

古地図の「めくれ」演出（通った場所だけくっきり見せる画像合成）も同様に、前回合成済みの画像を土台にして新区間だけ追加で描き足す方式に変更しました。どちらも、GPS 更新 1 回あたりの計算量が「これまで歩いた距離」ではなく「前回からの移動量」だけに依存するようになり、歩行時間が伸びても体感速度が変わらなくなります。

もう 1 つ、地図をズーム・パンした直後に古地図のテクスチャが消えて見える GPU 側の不具合への対策（オーバーレイをいったん外して貼り直す）も、当初はカメラが止まるたび毎回実行していました。これはチェックポイントのマーカーを選択して開いた名前表示まで巻き添えで閉じてしまう副作用があり、「チェックポイントをタップしても、名前がすぐ消える」という別のバグとして報告されました。原因は、貼り直し処理がマーカーの.map をいったん nil にしてから戻す実装になっており、これが Google Maps SDK 上でマーカーの選択状態を解除してしまうためでした。ズームが一定以上変化した時だけ貼り直すよう条件を絞ることで解決しています。

この事例から見える教訓は、「毎回全部やり直す」実装は、データ量が小さいうちは正しく動いて見えてしまう、という点です。個人開発では、自分自身が最初の（そして最も厳しい）テスターです。数分の動作確認では気づけない劣化は、実際に長時間使ってみる、あるいはコードを読み返して「この処理は入力サイズに対してどう振る舞うか」を自問する習慣でしか見つけれられません。

サインイン後の Web 版体験も見直した

以前はサインイン後の「時空旅」タブを自分の記録だけに絞っていましたが（第 20 章のティール型戦略で触れた「独占ニッチ」を強く意識しすぎた設計でした）、実際に使ってみると「サインインすると、それまで見えていたみんなの時空旅が見えなくなる」という体験の分断が気になり、自分の記録と他ユーザーの公開記録を再び一緒に一覧できるように戻しました。公開されている記録には🌐アイコンを付け、「これは誰でも見られる記録だ」と分かるようにしています。機能の追加・削除は一直線には進まず、実際に使ってみて「行き過ぎた」と感じたら素直に戻す判断も、個人開発の大事な一部です。

22-6. 旅行記機能：既存の AI 生成資産を再利用する設計

歩いた記録を「その場で振り返る」体験をさらに一步進め、時間旅の詳細画面に「旅行記を作成する」ボタンを追加しました。旅の名前・使った古地図・巡った御朱印スポット・投稿写真・ユーザー自身の感想を材料に、AI がエッセイ調の旅行記を Markdown 形式で書き上げ、生成後は同じ画面から「旅行記を読む」ボタンでいつでも読み返せます。

新しい AI 呼び出しを増やす前に、既にある生成物を探す

このアプリには、史跡チェックポイントを開くと「その場所の昔の物語」を AI が生成し、CheckpointStory として端末に保存しておく仕組みが既がありました（同じ場所を何度開いても再度 AI に問い合わせない、という利用料対策の設計です）。旅行記生成でも、巡った御朱印スポットそれぞれについて安易に新しい AI 呼び出しを追加するのではなく、まず「その場所の詳細は既に生成済みか」を CheckpointStory から確認し、あればそれをほぼそのまま旅行記に取り込み、無い場合だけその場で生成してキャッシュに保存する設計にしました。

```
if let existing = fetchSavedStory(siteID: site.id, modelContext: modelContext) {
    details.append((site: site, title: existing.title, body: existing.body))
} else {
    let story = try await historyService.generateStory(for: site.coordinate, overlayMap: overlayMap, placeName: site.name)
    modelContext.insert(CheckpointStory(siteID: site.id, title: story.title, body: story.body))
}
```

この設計には 2 つの利点があります。1 つは単純に API 利用料と待ち時間の節約です。もう 1 つは、**アプリの別の画面（チェックポイント詳細シート）と旅行記とで、同じ場所について書かれる内容が一致する**という体験上の一貫性です。プロンプト側にも「取得済みの文章は要約・言い換えをしない」という指示を明示的に与え、AI が同じ場所について毎回違う言い回しを作り出してしまわないようにしています。既存の機能を作るときに用意した「一度生成したら再利用する」という設計判断は、後から生えてくる新機能にとっても資産になる、という一例です。

AI 呼び出しはどこで行うか——クライアント側かサーバー側か

旅行記生成の AI 呼び出しは、既存の物語生成（AIHistoryService）と同じく、iOS クライアントから直接 OpenAI API を叩く方式を踏襲しました。Cloud Functions 経由でサーバー側から AI API キーを使う設計（キーを端末に持たせず、利用量の管理もしやすい）も選択肢としてはありましたが、この機能のためだけに新しい Cloud Functions のデプロイ・シークレット管理を増やすコストと、既存の設定画面・Keychain 保存の仕組みをそのまま使える手軽さを比べ、後者を選びました。機能ごとに

「一貫性を取るか、その機能に最適な設計を取るか」を都度判断する必要があり、今回は既存パターンの一貫性を優先しています。

文章だけでは物足りない——スクラップブック形式への作り直し

最初のバージョンは、AI が書いたエッセイをそのまま 1 本の Markdown として表示するだけでした。実際に読んでみると、文章の中に「桜田門を過ぎ、和田倉門へ向かった」と書かれていても、対応する御朱印の写真や歩いたルートの地図がすぐ横に見えないと、記録としての実感が薄いことに気づきました。そこで旅行記の画面を、AI の文章に加えて、実際に歩いたルートの地図・巡った御朱印（写真＋その場所の説明）・投稿写真（写真＋説明）を並べた、スクラップブックのような構成に作り直しました。

```
JournalGalleryRow(  
  image: stamp.photo,  
  title: site.name,  
  detail: checkpointDetail(siteId: site.id) ?? site.summary  
)
```

ここでのポイントは、この「写真＋説明」の一覧を新しいデータ構造として別途持たせるのではなく、御朱印なら CheckpointStory、投稿写真なら WalkPhotoPost.storyBody という、**既にアプリの中に存在する情報をそのまま並べ直した**だけという点です。旅行記という新しい画面のために新しいデータを追加で作り込む必要はなく、既存のデータを「1 つの画面にまとめて見せる」ビューを 1 つ書けば済みました。

生成した内容を、そのまま Web 版にも流す

旅行記の本文（Markdown）は WalkRoute に保存され、既存の同期の仕組み

（users/{uid}/walkRoutes・公開時は sharedTrips）にフィールドを 2～3 個足すだけで、Web 公開ページ（komap.ktrips.net）側にもそのまま伝わるようにしました。Web 側では新しく AI を呼び出したり、内容を作り直したりはせず、iOS で生成済みの Markdown 本文を、Kindle 原稿プレビューで使っているのと同じ marked による Markdown→HTML 変換にそのまま渡すだけです。

```
const journalHtml = marked.parse(trip.journalMarkdown, { async: false }) as string;
```

写真＋説明のスクラップブック表示についても、Web 版で同じ体験を再現したくなりますが、Web 側は普段 iOS 端末上にしかない CheckpointStory（御朱印スポットの説明）を直接読むことができません。ここで新しい Firestore コレクションを作って同期の仕組みを 1 つ増やすことも検討しましたが、既に「みんなの時空旅」を公開する際に御朱印・投稿写真の画像をコピーして sharedTrips に載せている処理（setPubliclyShared）があったため、そこに説明文（detail）を 1 フィールド足して一緒にコピーする方式にとどめました。公開のタイミングで一度だけ複製される非正規化したデータですが、「新しい同期の仕組みを 1 つ増やす」より「既にある同期処理にフィールドを 1 つ足す」方が、変更範囲もテストする経路も小さく済みます。

「1 箇所で生成し、複数のチャンネルにそのまま流用する」という発想は、22-4 で触れた Kindle 原稿プレビューの配信方法とも共通しています。新しいコンテンツの置き場所を増やすたびに、そこでしか読めない専用の生成ロジックを作るのではなく、**単一の生成物を複数の画面・チャンネルで使い回せ**

るように設計しておく、後から表示先を増やす作業がとても軽くなります。

リリース後も、細かいフィードバックを1つずつ形にしてい

旅行記機能は、最初の実装で終わりではありませんでした。実際に使ってみて出てきた具体的な要望を、その都度小さな修正として積み重ねています。

- AI が書く本文が、各史跡の詳細を丸ごと引用する長いエッセイだったため、御朱印・写真のギャラリー側と内容が重複して読みにくくなっていました。プロンプトを書き直し、個々の詳細はギャラリー側だけに任せて、本文は旅全体の印象を 200 字程度でまとめる「旅のサマリー」に絞りました
- 「時空旅を開いた時の画面」と「旅行記画面」で、名前・公開状況・日付・距離・件数といった同じ種類の情報の並び順がバラバラだったため、両方のヘッダーを完全に同じ構成・同じ並び順に揃えました（その後、旅行記画面だけ名前横の古地図名を外し、代わりにサマリー見出し「（古地図名）の時空旅」へ意図的に移すという細部の作り分けも行っていますが、これは揃えた後だからこそできた「意図した差分」です）
- Web 版は当初、地図・御朱印ギャラリー・投稿写真ギャラリーが「一般的な一覧セクション」と「旅行記セクション」の 2 箇所重複して存在していました（旅行記の有無に関わらず表示される簡易版と、旅行記がある時だけ追加される詳細版）。地図を 2 つ読み込む無駄も含め、これを 1 本の流れに統合し、旅行記の有無に関わらず同じ構成で表示するようにしました
- 「みんなの時空旅」を未サインインで開いた訪問者に画像が表示されない不具合も見つけました。原因は Firebase Storage のルールで、対応する Firestore ドキュメントは公開読み取り可にしていたのに、画像本体（Storage 側）だけサインイン必須のままになっていました。同じ「公開」という状態を、Firestore と storage の 2 箇所のルールで別々に管理していたために、片方だけ直し忘れていたのです。関連するリソースの公開範囲は、片方を変えたらもう片方も揃っているか必ず確認する癖が要ります
- 個別の時空旅を `?trip=<id>` という URL で直接開けるようにもしました。ルーティングライブラリを新規に入れるほどの規模ではなかったため、URLSearchParams と history.pushState だけを使う小さなカスタムフック（useSelectedTripId）を 1 つ書き、選択状態と URL を双方向に同期させています。「共有できる URL が無い」ことは機能不足には見えにくいものですが、実際には「この時空旅、誰かに見せたい」という一番自然な欲求を塞いでいました
- 公開ページ右上の「使い方」ボタンは、個別の時空旅を開いている間はサインインの案内（ページ上部の案内文）が隠れてしまうため、その間だけ「ログイン」ボタンに差し替えました。ボタンの色も Google サインインボタンと揃え、「今何をすべきか」が一目で分かるようにしています
- 「今 N 人が時空旅中」という閲覧人数の表示は、当初ヘッダー中央に単独で置いていましたが、Komap のアプリアイコン・サービス名のすぐ右横（一覧表示時のみ）に移し、「アイコ

ン→サービス名→閲覧人数」の順に揃えました。関連する情報同士を近づけるだけで、「今 Komap を使っている人がこれだけいる」という文脈が伝わりやすくなります

- 個別の時空旅を選んで詳細を表示している間は、一覧側の「N 件の旅」という件数表示を隠すようにしました。今見ている内容と無関係な数字をヘッダーに残しておくより、空いた分だけ右側の要素（サインインボタン等）を右へ寄せた方が、限られた横幅を有効に使えます
- 日時の表記が画面ごとにバラバラ（「2026 年 9 月 10 日」「9/10 14:30」など）になっていたため、YYYY/M/D HH:MI という 1 つの形式に統一しました。地味な変更ですが、複数の画面を行き来するユーザーにとって表記の揺れは「作り込みの甘さ」として伝わりやすい部分です
- TestFlight への招待送信が Apple の API 仕様上の理由（STATE_ERROR）で失敗した時、以前は技術的なエラーコードをそのまま表示していましたが、ユーザーには「招待の準備に問題が発生しました」という平易な案内に変え、代わりに開発者（管理者）宛にその場で報告メールを送れる mailto:リンクを添えました。ユーザーに不要な技術詳細を見せず、それでも問題を報告する手段は残しておくバランスです

いずれも機能としては地味な調整ですが、「同じ情報は同じ場所・同じ順で見せる」「重複した実装を 1 つにまとめる」「関連する設定は揃っているか確認する」「画面の状態によって、今ユーザーが一番押してほしい導線を出し分ける」という、当たり前だけれど気を抜くと崩れる原則の積み重ねです。個人開発では大きな新機能よりも、こうした小さな揃え直しの数のほうが、実際の使い心地に効いてきます。

22-7. 本章から学べること

- 外部機器（IoT デバイス等）との連携は、相手側の API 仕様が事前の想定と異なることが多い。「送信しました」で終わらず、HTTP ステータスやエラー内容を画面に出し、ユーザー自身が原因を切り分けられるようにしておく
- SwiftData の@Query を活用すると、詳細画面の中でデータを編集・削除しても一覧が自動的に追従する。手動で配列を管理し直す必要がなくなり、削除後の状態管理のバグを減らせる
- ゲーム性のための演出（ぼかし・フェード等）は、実用性を損なわない範囲に収まっているか、リリース後の実際のフィードバックで必ず検証し、数値を追いつめ直す
- Web 版・Kindle 原稿・SNS など、複数のチャネルすべてを「本体験（iOS アプリ）への入り口」として一貫して設計すると、個別のチャネルが小さくても全体としての導線が強くなる
- 「動いているから十分」で終わらず、無関係な状態変化が重い処理を誤って引き起こしていないか（React の依存配列、SwiftUI の再描画）を定期的に疑う。ユーザーからの「遅い」という一言は、たいてい具体的な 1 箇所の実装に原因があり、計測すれば必ず見つかる

- 「入力が増えるたびに全体をやり直す」実装は、テスト中の小さなデータ量では問題なく動いて見えてしまう。ループ処理を書くたびに「これは蓄積したデータ量に比例して重くなる処理か、前回からの差分だけで済む処理か」を一呼吸置いて確認する習慣が、リリース後の劣化を防ぐ
- 機能を絞り込む判断（ティール型の「独占ニッチ」戦略など）は強力だが、絶対ではない。実際の利用感で「絞りすぎた」と感じたら、素直に体験を戻す柔軟さも持つておく
- 新しい AI 機能を追加する前に、既に生成済みのデータを再利用できないか（キャッシュ・他画面の生成物）を先に確認する。利用料・待ち時間の節約になるだけでなく、画面をまたいだ内容の一貫性という体験上の利点にもつながる
- 「公開」のような 1 つの概念を、Firestore のルール・Storage のルールのように複数の場所で別々に判定していると、片方だけ直して片方を直し忘れる不具合が起きやすい。関連する設定を変更する時は、同じ概念を判定している他の場所がないか一緒に洗い出す
- 同じ情報（名前・日付・件数など）を複数の画面に出す時は、項目の並び順まで揃えておく。並び順がバラバラだと、ユーザーは画面ごとに情報を探し直す必要が生まれ、地味に体験を損なう
- 「個別のコンテンツを URL で直接開けるか」は機能一覧には現れにくいですが、シェアされて初めて新規訪問者が増える Web 公開ページでは効果が大きい。ルーティングライブラリを丸ごと導入しなくても、URLSearchParams と history.pushState だけの小さなフック 1 つで十分なことも多い
- ヘッダーのボタンのような「常に同じ場所にある操作」も、画面の状態（一覧か詳細か、サインイン済みか等）によって今ユーザーに一番押してほしい導線に出し分けると、案内文が隠れているのに導線も無い、という取りこぼしを防げる

付録 A 用語集

- **GMSGroundOverlay** : Google Maps SDK で、指定した緯度経度範囲に画像を貼り付けるためのクラス
- **ジオリファレンス** : 地図画像に対して、実世界の座標（緯度経度）を対応付ける作業
- **テクスチャアトラス** : GPU 上で複数の画像をまとめて扱うための領域。上限を超えると描画不具合が起きる
- **Chaikin のコーナーカット法** : 折れ線の角を滑らかにするアルゴリズム
- **SwiftData** : iOS 17 以降の標準永続化フレームワーク
- **Firestore** : Firebase が提供する NoSQL 型のリアルタイムデータベース
- **WCSession** : iPhone と Apple Watch 間の通信を担う WatchConnectivity フレームワークの中核クラス
- **HKWorkoutSession** : HealthKit でワークアウト（運動）セッションを管理するクラス
- **ASO(App Store Optimization)** : App Store 内検索での見つけやすさを高める最適化施策
- **UGC(User Generated Content)** : ユーザーが生成するコンテンツ（投稿写真、コメント等）
- **社会的証明 (Social Proof)** : 他者の行動・評価が自分の意思決定に影響を与える心理効果

付録 B 開発チェックリスト

金曜夜を迎える前（平日のうちに済ませる事前準備） - ☐ Google Cloud Platform アカウントを作成し、Google Maps API キーを発行した - ☐ Apple Developer Program への登録・年会費の支払いを完了した（本人確認に数日かかることがあるため、この週末にリリースする場合は前の週のうちに必須） - ☐ Firebase プロジェクトを作成した - ☐ AI API（OpenAI 等）のアカウントを作成し、API キーを発行した - ☐ コア MVP のスコープ（第 2-3 節）を紙に書き出し、削る機能・残す機能を決めた

設計・企画 - ☐ 誰が、どんな場面で使うアプリかを一文で説明できる - ☐ 地図に重ねるレイヤーの種類（古地図／オリジナルマップ等）を決めた - ☐ 収集要素（御朱印／バッジ等）の設計を決めた

技術基盤 - ☐ XcodeGen で project.yml を管理し、.xcodproj は.gitignore 済み - ☐ Google Maps API キーを.xcconfig で管理し、Bundle ID 制限をかけた - ☐ 位置情報の Usage Description をすべて用意した - ☐ HealthKit を使う場合、Update 用の説明文も用意した

セキュリティ - ☐ Firestore セキュリティルールで「本人のみ書き込み可」を徹底した - ☐ API キー・サービスアカウントの秘密鍵を Git にコミットしていない - ☐ 画像アップロード前にリサイズ・圧縮している

リリース前 - ☐ TestFlight の内部テストで実地テストを行った - ☐ 外部テストグループにビルドを

追加し、ベータ版 App Review の承認を得た（第 12-2-1 節） - ☐ App Privacy の申告内容が実際の実装と一致している - ☐ プライバシーポリシーを公開 URL として用意した - ☐ スクリーンショット・プレビュー動画で「体験」を伝えられている - ☐ App Store Connect で Submit for Review を実行し、審査待ちの状態になっている（第 12-5 節の月曜早朝チェックリスト）

マーケティング - ☐ SNS で Before/After コンテンツを準備した - ☐ リリース日を告知し、初速を作る施策を用意した - ☐ Firebase Analytics で主要指標を計測できる状態にした

付録 C 参考リソース

- Google Maps Platform 公式ドキュメント（Maps SDK for iOS）
- Apple Developer Documentation（CoreLocation、HealthKit、WatchConnectivity、SwiftData、StoreKit）
- Firebase 公式ドキュメント（Authentication、Firestore、Storage、Cloud Hosting）
- OpenAI Platform ドキュメント（Chat Completions API）
- XcodeGen 公式リポジトリ（GitHub: yonaskolb/XcodeGen）
- Wikimedia Commons（パブリックドメインの歴史地図史料）

OpenStreetMap（地図タイルデータ、ODbL ライセンス）